

软件安全作业三 —— 思考题

班级：2022211801 学号：2022211570 姓名：项枫

一、测试

1、CWE121_Stack_Based_Buffer_Overflow__CWE805_wchar_t_declare_snprintf_05.c

测试代码：

```
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if(staticTrue)
    {
        data = dataBadBuffer;
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100-1);
        source[100-1] = L'\0';
        SNPRINTF(data, 100, L"%s", source);
        printWLine(data);
    }
}
```

运行结果：运行

test01/CWE121_Stack_Based_Buffer_Overflow__CWE805_wchar_t_declare_snprintf_05.c:



结果分析:

第 1 点指出了缓冲区溢出风险。

修改代码, data 指针初始化为 dataGoodBuffer (大小为 100):

```
data = dataGoodBuffer;
```

询问这样的修改是否能避免这个问题:

启发式软件安全编程助手 v1

新对话

/question 修改代码...

已加载全部

/question 修改代码, data指4条记录qpt-4o-mini

/question
修改代码, data 指针初始化为 dataGoodBuffer (大小为 100) :
data = dataGoodBuffer;
请问这样的修改是否能避免缓冲区溢出问题。

修改代码分析: data 指针指向 dataGoodBuffer

修改后的代码片段

```
c
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if(staticTrue)
    {
        data = dataGoodBuffer; // <--- 修改: 指向较大缓冲区
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100-1);
    }
}
```

输入问题, 发送 [Enter] 换行 [Ctrl(Alt/Shift) + Enter]

启发式软件安全编程助手 v1

新对话

/question 修改代码...

已加载全部

/question 修改代码, data指4条记录qpt-4o-mini

```
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if(staticTrue)
    {
        data = dataGoodBuffer; // <--- 修改: 指向较大缓冲区
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100-1);
        source[100-1] = L'\0';
        SNPRINTF(data, 100, L"%s", source);
        printWLine(data);
    }
}
```

是否避免了缓冲区溢出问题?

部分避免, 但仍存在潜在风险

- 直接问题解决: 通过将 data 指针指向 dataGoodBuffer (大小为 100), 确实避免了原始的缓冲区溢出问题, 因为现在 data 的容量足够容纳 SNPRINTF operation 指定的 100 个字符。
- 残余风险:
 - SNPRINTF 的大小参数: 函数中 SNPRINTF(data, 100, L"%s", source); 指定了大小为 100, 但这没有考虑终止空字符 (NUL)。在标准 C 库中, 字符串函数通常计入终止符, 因此为容纳 terminated 字符串, 缓冲区通常应多预留一个字符空间。
 - 源数据 source 潜在溢出: 虽然 dataGoodBuffer 有足够的空间, 但如果 source 数组在后续操作中被修改为超过 100 个字符 (不计终止符), 或如果 wmemset 写入的字符数量参数发生错误, 仍可能导致溢出。

输入问题, 发送 [Enter] 换行 [Ctrl(Alt/Shift) + Enter]



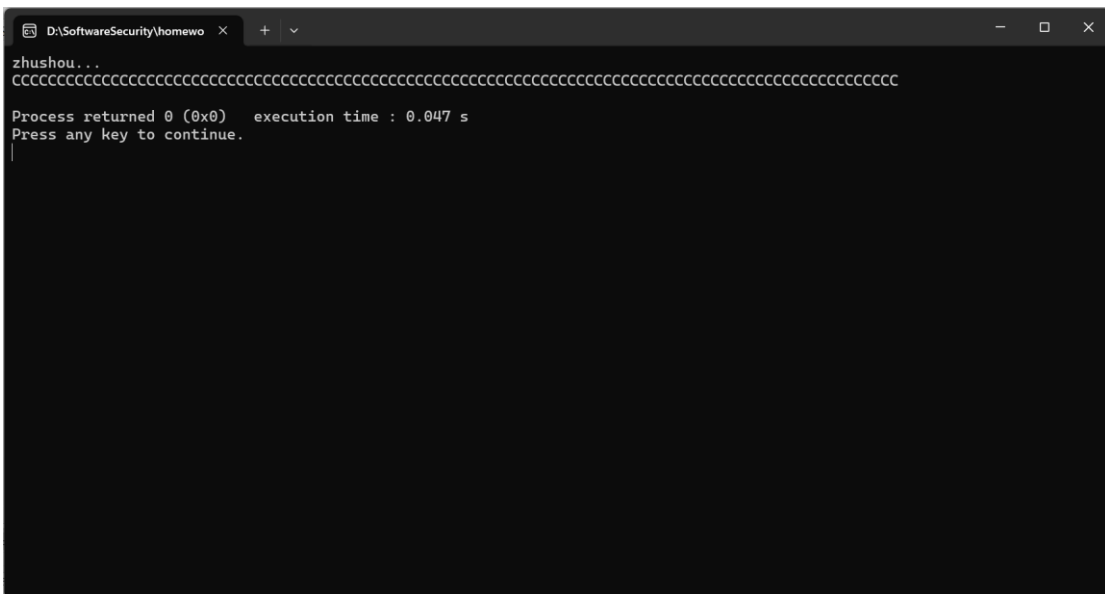
系统回复这样的修改成功避免了这个问题，同时给出了一些其他建议，如 SNPRINTF 的大小参数等。

修改后的代码：

```
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if (staticTrue)
    {
        data = dataGoodBuffer; // <--- 修改：指向较大缓冲区
        data[0] = '\0';
    }
    {
        wchar_t source[100];
        memset(source, 'C', 100-1);
        source[100-1] = '\0';
        SNPRINTF(data, 100, \"%s\", source);
        printWLine(data);
    }
}
```

代码运行结果：运行 test01/zhushou.c：

运行结果正确。



(2) ChatGPT

测试输入：



测试输出：

漏洞分析

1. 数据缓冲区溢出漏洞:

- 在以下代码中:

```
cpp
data = dataBadBuffer;
```

`data` 被设置为 `dataBadBuffer`, 其大小为 50.

- 然后, 在调用 `SNPRINTF` 时:

```
cpp
SNPRINTF(data, 100, L"%s", source);
```

这里尝试将 `source` (大小为 100) 写入 `data` (大小为 50), 这会导致缓冲区溢出.

- 在 `dataBadBuffer` 的大小是 50 字节时, 向 `data` 写入 100 字节的数据 (由 `source` 提供) 会导致缓冲区溢出, 从而可能引发未定义行为 (如覆盖其他内存、造成程序崩溃等).

2. 使用 `SNPRINTF` 函数时可能存在的风险:

- 即使在代码中已经设置了 `SNPRINTF` 的大小限制 (100), 但是如果 `data` 指向的是一个较小的缓冲区 (如 `dataBadBuffer`), 那么依然会发生缓冲区溢出.

修复建议

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

修复后的代码:

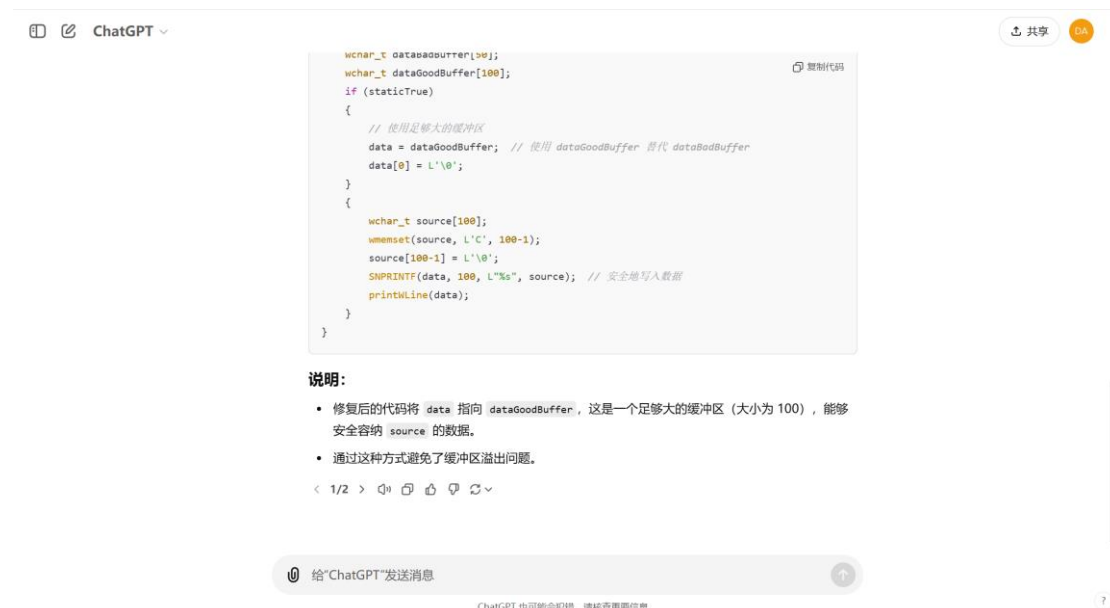
```
cpp
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if (staticTrue)
    {
        // 使用足够大的缓冲区
        data = dataGoodBuffer; // 使用 dataGoodBuffer 替代 dataBadBuffer
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100-1);
        source[100-1] = L'\0';
        SNPRINTF(data, 100, L"%s", source); // 安全地写入数据
        printLine(data);
    }
}
```

说明:

- 修复后的代码将 `data` 指向 `dataGoodBuffer`, 这是一个足够大的缓冲区 (大小为 100), 能够

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析：

该模型找到了测试代码的缓冲区溢出漏洞并提供了漏洞所在行：

```
data = dataBadBuffer;
SNPRINTF(data, 100, L"%s", source);
```

模型修改的代码：

```
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if (staticTrue)
    {
        // 使用足够大的缓冲区
        data = dataGoodBuffer; // 使用 dataGoodBuffer 替代 dataBadBuffer
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100-1);
        source[100-1] = L'\0';
        SNPRINTF(data, 100, L"%s", source); // 安全地写入数据
        printWLine(data);
    }
}
```

代码运行结果：运行 test01/chatgpt.c:

运行结果正确。

[illegible]

2、CWE78_OS_Command_Injection__char_file_w32_execlv_06.c

测试代码：

```
void function()
{
    char * data;

    char dataBuffer[100] = COMMAND_ARG2;

    data = dataBuffer;

    if (STATIC_CONST_FIVE==5)
    {
        {
            size_t dataLen = strlen(data);

            FILE * pFile;

            if (100-dataLen > 1)
            {
                pFile = fopen(FILENAME, "r");

                if (pFile != NULL)
                {
                    if (fgets(data+dataLen, (int)(100-dataLen), pFile) == NULL)
                    {
                        printLine("fgets() failed");

                        data[dataLen] = '\0';
                    }

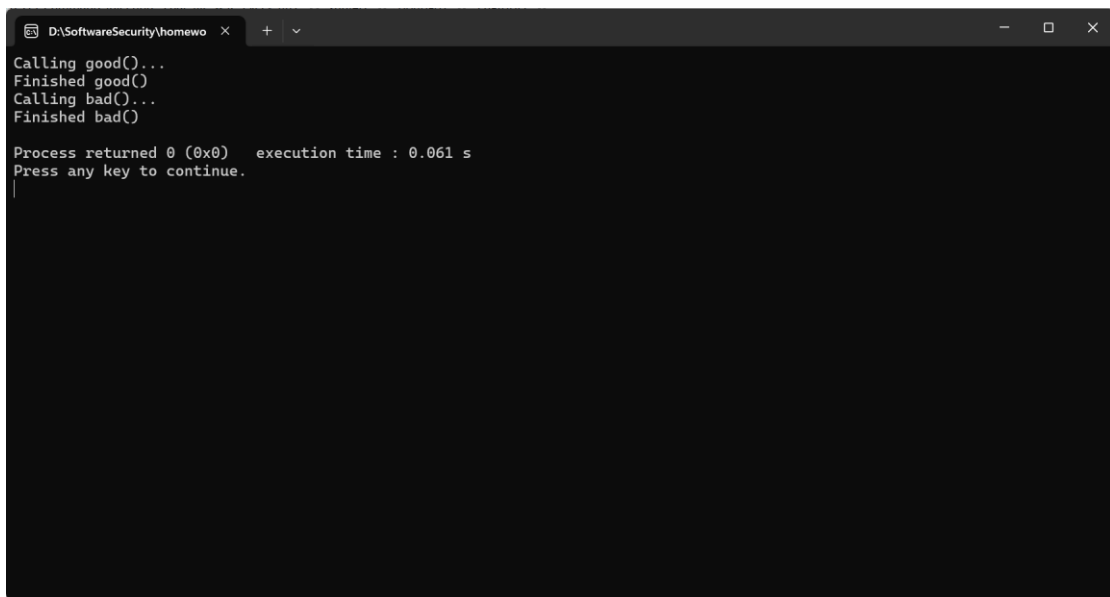
                    fclose(pFile);
                }
            }
        }
    }
}
```

```

    }
}
}
{
    char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
    EXECV(COMMAND_INT_PATH, args);
}
}

```

运行结果：运行 test02/ CWE78_OS_Command_Injection__char_file_w32_execv_06.c:



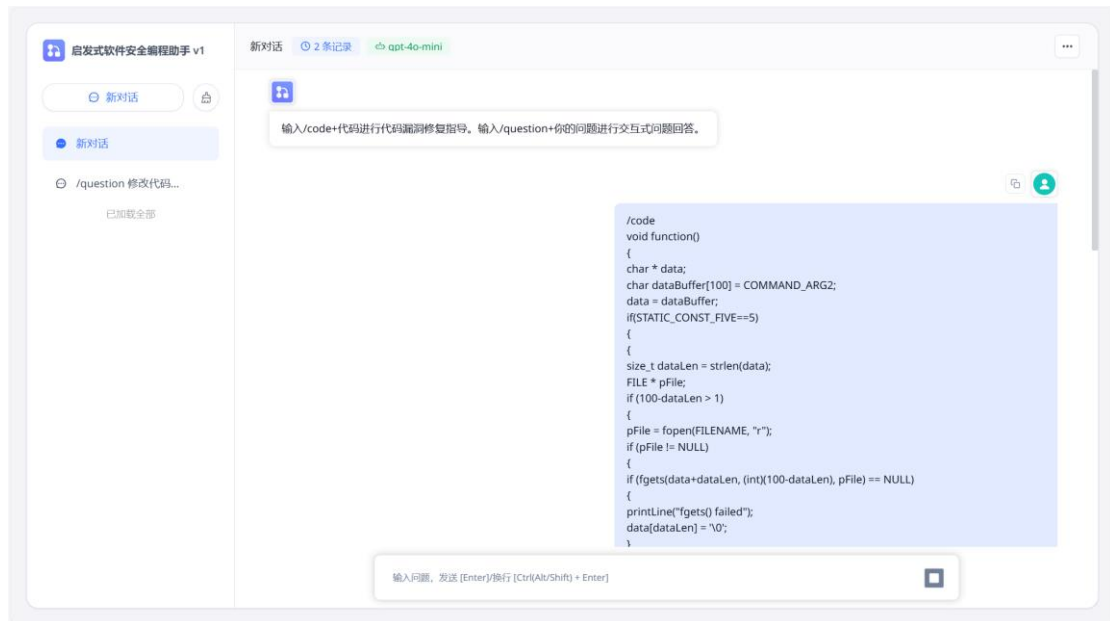
```

D:\SoftwareSecurity\homewo x + v
Calling good()...
Finished good()
Calling bad()...
Finished bad()
Process returned 0 (0x0) execution time : 0.061 s
Press any key to continue.

```

(1) 启发式软件安全编程助手

测试输入:



测试输出：





结果分析:

第 3 点指出了 `WXWCV` 调用问题。

修改代码:

```
if(STATIC_CONST_FIVE==5)
{
    strcat(data, ".");
}
```

询问这样的修改是否能避免这个问题:





修改后的代码：

```
void function()
{
    char * data;
    char dataBuffer[100] = COMMAND_ARG2;
    data = dataBuffer;
    if(STATIC_CONST_FIVE==5)
    {
        strcat(data, "*,*");
    }
    {
        char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
```

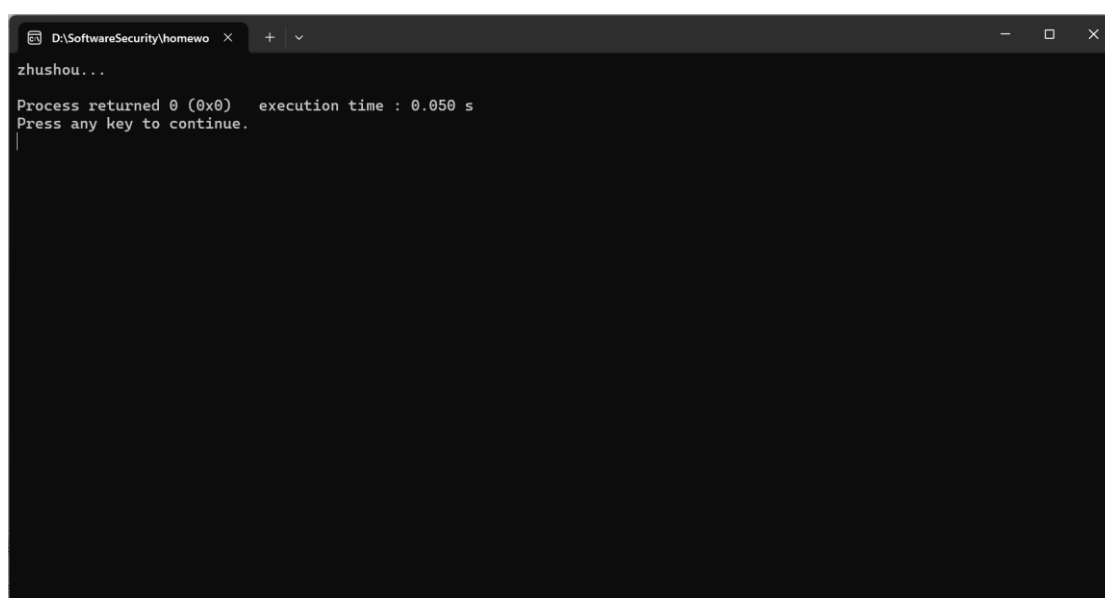
```

        EXECV(COMMAND_INT_PATH, args);
    }
}

```

代码运行结果：运行 test02/xunfei.c:

运行结果正确。



(2) ChatGPT

测试输入:



测试输出:

漏洞分析

1. 缓冲区溢出:

- 这段代码涉及缓冲区 `dataBuffer`，并使用 `fgets` 读取数据到 `data`：

```
cpp
size_t dataLen = strlen(data);
if (100 - dataLen > 1)
{
    pFile = fopen(FILENAME, "r");
    if (pFile != NULL)
    {
        if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
        {
            printf("fgets() failed");
            data[dataLen] = '\0';
        }
        fclose(pFile);
    }
}
```

- `dataBuffer` 的大小是 100 字节，如果 `fgets` 从文件中读取的数据超过 100 字节时，将会导致缓冲区溢出。虽然你控制了读取长度 `100 - dataLen`，但仍需确保 `data` 指向的缓冲区大小足够。

2. 文件路径未验证:

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

2. 文件路径未验证:

- `FILENAME` 是文件路径的变量，但没有对其进行验证。如果这个路径是由外部输入提供的，可能会导致路径遍历攻击，或者读取敏感文件。因此，需要验证或过滤文件路径。

3. 命令注入风险:

- 在执行命令时，使用了 `COMMAND_INT_PATH` 和其他命令参数：

```
cpp
char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
EXECV(COMMAND_INT_PATH, args);
```

- 如果这些命令路径或参数来自不受信任的输入，可能存在命令注入的风险，攻击者可能利用这些输入执行任意命令。

修复建议

1. 确保缓冲区大小正确:

- 要保证 `fgets` 读取的数据不会导致缓冲区溢出。可以通过显式检查缓冲区的大小，并确保读取的内容不会超出其边界。

2. 验证文件路径:

- 对文件路径 `FILENAME` 进行验证，确保其合法性，防止路径遍历攻击。

3. 验证命令路径和参数:

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析:

该模型找到了测试代码的缓冲区溢出、命令注入漏洞并提供了漏洞所在行:

```
size_t dataLen = strlen(data);
if (100 - dataLen > 1)
{
    pFile = fopen(FILENAME, "r");
    if (pFile != NULL)
    {
        if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
        {
            printLine("fgets() failed");
        }
    }
}
```

```

        data[dataLen] = '\0';
    }
    fclose(pFile);
}
}

```

```

char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3,
NULL};EXECV(COMMAND_INT_PATH, args);

```

模型修改的代码：

```

void function()
{
    char dataBuffer[100] = COMMAND_ARG2;
    char * data = dataBuffer;

    if (STATIC_CONST_FIVE == 5)
    {
        size_t dataLen = strlen(data);
        FILE * pFile;
        // 确保不会发生缓冲区溢出
        if (100 - dataLen > 1)
        {
            pFile = fopen(FILENAME, "r");
            if (pFile != NULL)
            {
                if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
                {
                    printLine("fgets() failed");
                    data[dataLen] = '\0'; // 确保字符串以 \0 结束
                }
                fclose(pFile);
            }
        }
    }

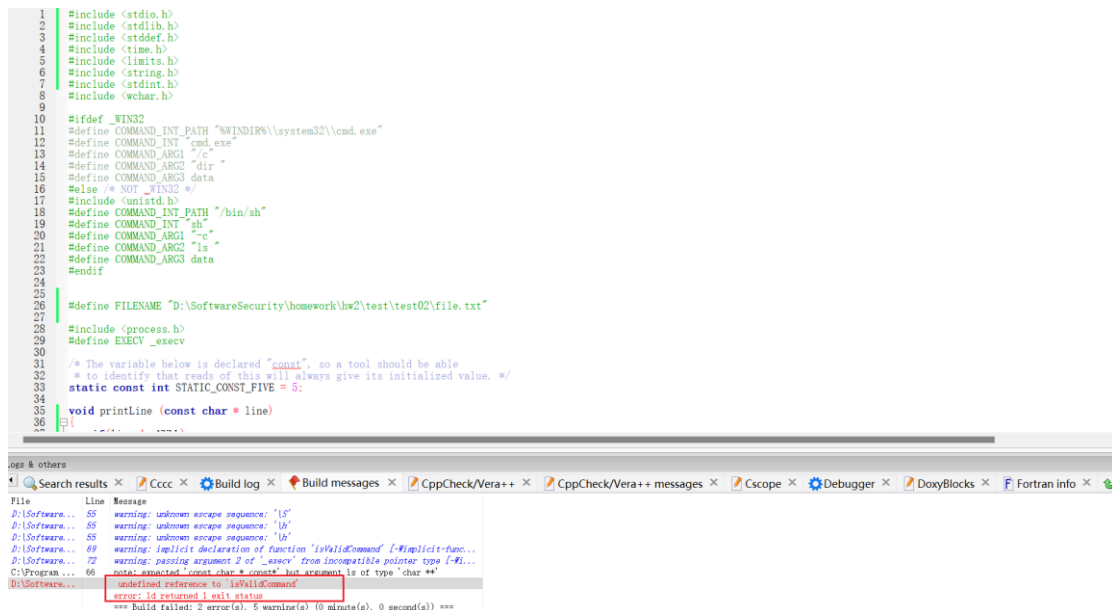
    // 验证命令路径和参数，确保不被注入
    if (isValidCommand(COMMAND_INT_PATH)) // 假设 isValidCommand 是检查命令路径的函数
    {
        char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
        EXECV(COMMAND_INT_PATH, args);
    }
}

```

代码运行结果：运行 test02/chatgpt.c:

代码运行失败:

```
||=== Build file: "no target" in "no project" (compiler: unknown) ===|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|In function 'function':|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|55|warning: unknown escape
sequence: '\S'|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|55|warning: unknown escape
sequence: '\h'|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|55|warning: unknown escape
sequence: '\h'|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|69|warning: implicit declaration
of function 'isValidCommand' [-Wimplicit-function-declaration]|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|72|warning: passing argument 2 of
'_execv' from incompatible pointer type [-Wincompatible-pointer-types]|
C:\Program Files\CodeBlocks\MinGW\x86_64-w64-mingw32\include\process.h|66|note: expected
'const char * const*' but argument is of type 'char **'|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.o:chatgpt.c|| undefined reference
to `isValidCommand'|
||error: ld returned 1 exit status|
||=== Build failed: 2 error(s), 5 warning(s) (0 minute(s), 0 second(s)) ===|
```



```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stddef.h>
4  #include <time.h>
5  #include <limits.h>
6  #include <string.h>
7  #include <stdint.h>
8  #include <wchar.h>
9
10 #ifdef _WIN32
11 #define COMMAND_INT_PATH "%WINDIR%\\system32\\cmd.exe"
12 #define COMMAND_INT "cmd.exe"
13 #define COMMAND_ARG1 "/c"
14 #define COMMAND_ARG2 "dir"
15 #define COMMAND_ARG3 data
16 #else /* NOT _WIN32 */
17 #include <unistd.h>
18 #define COMMAND_INT_PATH "/bin/sh"
19 #define COMMAND_INT "sh"
20 #define COMMAND_ARG1 "-c"
21 #define COMMAND_ARG2 "ls"
22 #define COMMAND_ARG3 data
23 #endif
24
25
26
27 #define FILENAME "D:\\SoftwareSecurity\\homework\\hw2\\test\\test02\\file.txt"
28
29 #include <process.h>
30 #define EXECV _execv
31
32 /* The variable below is declared "const", so a tool should be able
33  * to identify that reads of this will always give its initialized value. */
34 static const int STATIC_CONST_FIVE = 5;
35
36 void printline (const char * line)
37 {
38     printf("%s\n", line);
39 }
```

The screenshot of the CodeBlocks IDE shows the build error messages. The messages are:

- Warning: unknown escape sequence: '\S'
- Warning: unknown escape sequence: '\h'
- Warning: unknown escape sequence: '\h'
- Warning: implicit declaration of function 'isValidCommand' [-Wimplicit-function-declaration]
- Warning: passing argument 2 of '_execv' from incompatible pointer type [-Wincompatible-pointer-types]
- Note: expected 'const char * const*' but argument is of type 'char **'
- Undefined reference to 'isValidCommand'
- Error: ld returned 1 exit status
- Build failed: 2 error(s), 5 warning(s) (0 minute(s), 0 second(s))

3、CWE190_Integer_Overflow__char_fscanf_add_09.c

测试代码:

```
void function()
{
    char data;
    data = ' ';
```

```

if(GLOBAL_CONST_TRUE)
{
    fscanf (stdin, "%c", &data);
}
if(GLOBAL_CONST_TRUE)
{
    {
        char result = data + 1;
        printHexCharLine(result);
    }
}
}

```

运行结果：运行 test03\CWE190_Integer_Overflow__char_fscanf_add_09.c:

```

D:\SoftwareSecurity\homewo
Calling good()...
1
32
0b
03
03
Finished good()
Calling bad()...
1
32
Finished bad()

Process returned 0 (0x0)   execution time : 3.883 s
Press any key to continue.

```

(1) 启发式软件安全编程助手

测试输入:



测试输出：





结果分析:

第 1 点指出了缓冲区问题。

修改代码, 将 `data` 定义为一个小的非 0 值:

```
data=2
```

询问这样的修改是否能避免这个问题:





修改后的代码：

```
void function()
{
    char data;
    data = ' ';
    if(GLOBAL_CONST_TRUE)
    {
        data = 2;
    }
    if(GLOBAL_CONST_TRUE)
    {
        {
            char result = data + 1;
            printHexCharLine(result);
        }
    }
}
```

代码运行结果：运行 test03/xunfei.c:

代码可以正常运行，且输出结果与 good()一致，正确。

```
D:\SoftwareSecurity\homewo x + v - □ x
zhushou...
03
Process returned 0 (0x0) execution time : 0.047 s
Press any key to continue.
```

(2) ChatGPT

测试输入:

📄 📄 ChatGPT ▾

📄 共享



请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    char data;
    data = ' ';
    if(GLOBAL_CONST_TRUE)
    {
        fscanf (stdin, "%c", &data);
    }
    if(GLOBAL_CONST_TRUE)
    {
        {
            char result = data + 1;
            printHexCharLine(result);
        }
    }
}
```

测试输出:

漏洞分析

1. 格式化输入不受控:

- 在这段代码中, `fscanf` 读取用户输入的字符并将其存储在 `data` 变量中:

```
cpp
fscanf(stdin, "%c", &data);
```

- `fscanf` 使用 `%c` 格式化符号读取单个字符。然而, `stdin` 是一个来自用户输入的流, 这意味着输入的数据并未经过有效的验证。用户可能输入一个非法字符 (例如控制字符), 导致后续代码可能不按预期工作。

2. 未初始化的 `data` 变量:

- 虽然代码给 `data` 赋了一个初值 `' '` (空格), 但这个赋值仅在输入部分条件成立时才会被覆盖。如果程序在未读取输入的情况下执行 `fscanf`, `data` 可能含有无效或意外的值。

3. 潜在的未定义行为:

- 在以下行:

```
cpp
char result = data + 1;
```

`data` 被增加了 1, 这在大多数情况下是有效的, 但如果 `data` 是一个极端的字符值 (例如控制字符或值大于字符类型的最大值), 可能会导致未定义行为。由于 `data` 是从 `stdin`

给“ChatGPT”发送消息

ChatGPT 也可能犯错。请核查重要信息。

```
cpp
char result = data + 1;
```

`data` 被增加了 1, 这在大多数情况下是有效的, 但如果 `data` 是一个极端的字符值 (例如控制字符或值大于字符类型的最大值), 可能会导致未定义行为。由于 `data` 是从 `stdin` 输入的, 攻击者可以尝试输入不期望的字符 (例如, 负数或超过 `char` 类型最大值的数值)。

4. 没有处理输入错误:

- 没有对 `fscanf` 的返回值进行检查, 这可能导致潜在的输入错误未被检测到。如果输入格式不正确, `fscanf` 会失败并不会按预期行为继续执行。

修复建议

1. 确保输入数据是有效的:

- 应该验证 `fscanf` 的返回值, 以确保输入操作成功并且输入是有效的。

2. 处理潜在的字符溢出或异常值:

- 确保 `data` 在执行任何算术操作之前是有效的, 并且不会溢出 `char` 类型的范围。

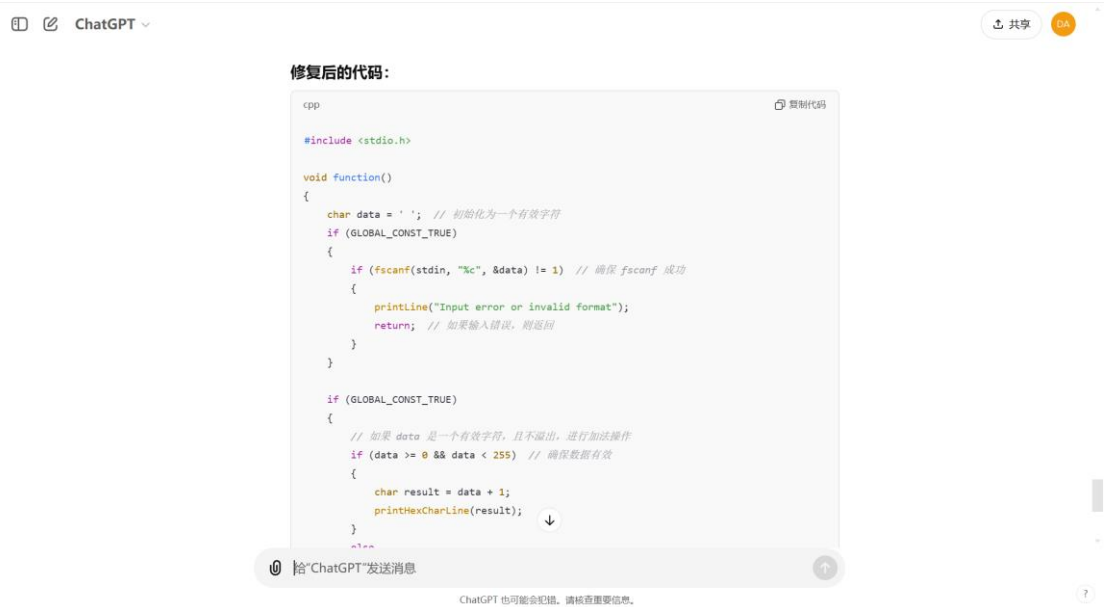
3. 检查输入的边界:

- 如果输入不符合预期, 可以提供反馈并要求重新输入。

修复后的代码:

给“ChatGPT”发送消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析:

该模型找到了测试代码的格式化输入不受控漏洞并提供了漏洞所在行:

```
fscanf(stdin, "%c", &data);
```

模型修改的代码:

```
void function()
{
    char data = ' '; // 初始化为一个有效字符
    if (GLOBAL_CONST_TRUE)
    {

```

```

    if (fscanf(stdin, "%c", &data) != 1) // 确保 fscanf 成功
    {
        printLine("Input error or invalid format");
        return; // 如果输入错误，则返回
    }
}

if (GLOBAL_CONST_TRUE)
{
    // 如果 data 是一个有效字符，且不溢出，进行加法操作
    if (data >= 0 && data < 255) // 确保数据有效
    {
        char result = data + 1;
        printHexCharLine(result);
    }
    else
    {
        printLine("Invalid character value");
    }
}
}

```

代码运行结果：运行 test03/chatgpt.c:

代码可以正常运行，但输出结果与 bad()一致。

```

D:\SoftwareSecurity\homewo > chatgpt...
1
32
Process returned 0 (0x0)   execution time : 2.291 s
Press any key to continue.

```

4、CWE78_OS_Command_Injection__wchar_t_environment_system_03.c

测试代码：

```

void function()
{
    wchar_t * data;
    wchar_t data_buf[100] = FULL_COMMAND;
    data = data_buf;
    if(5==5)
    {
        {
            size_t dataLen = wcslen(data);
            wchar_t * environment = GETENV(ENV_VARIABLE);
            if (environment != NULL)
            {
                wcsncat(data+dataLen, environment, 100-dataLen-1);
            }
        }
    }
    if (SYSTEM(data) != 0)
    {
        printLine("command execution failed!");
        exit(1);
    }
}

```

运行结果：运行

test04\CWE78_OS_Command_Injection__wchar_t_environment_system_03.c:

```
D:\SoftwareSecurity\homewo x + v
Calling good()...
驱动器 D 中的卷是 Data
卷的序列号是 6E99-8D8D

D:\SoftwareSecurity\homework\hw2\test\test04 的目录
2024/11/12 11:28 <DIR> .
2024/11/08 15:53 <DIR> ..
2024/11/12 11:27 1,346 chatgpt.c
2024/11/12 11:27 55,791 chatgpt.exe
2024/11/12 11:27 1,831 chatgpt.o
2024/11/12 11:23 3,193 CWE78_OS_Command_Injection__wchar_t_environment_system_03.c
2024/11/12 11:28 57,582 CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe
2024/11/12 11:28 2,879 CWE78_OS_Command_Injection__wchar_t_environment_system_03.o
2024/11/12 11:26 1,600 doubao.c
2024/11/12 11:28 56,656 doubao.exe
2024/11/12 11:28 1,950 doubao.o
2024/11/05 23:01 2,968 test04.txt
2024/11/12 11:23 1,518 xunfei.c
2024/11/12 11:24 57,164 xunfei.exe
2024/11/12 11:24 2,040 xunfei.o
13 个文件 246,518 字节
2 个目录 98,941,857,792 可用字节
驱动器 D 中的卷是 Data
卷的序列号是 6E99-8D8D

D:\SoftwareSecurity\homework\hw2\test\test04 的目录
2024/11/12 11:28 <DIR> .
2024/11/08 15:53 <DIR> ..
2024/11/12 11:27 1,346 chatgpt.c
2024/11/12 11:27 55,791 chatgpt.exe
2024/11/12 11:27 1,831 chatgpt.o
2024/11/12 11:23 3,193 CWE78_OS_Command_Injection__wchar_t_environment_system_03.c
2024/11/12 11:28 57,582 CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe
2024/11/12 11:28 2,879 CWE78_OS_Command_Injection__wchar_t_environment_system_03.o
2024/11/12 11:26 1,600 doubao.c
2024/11/12 11:28 56,656 doubao.exe
2024/11/12 11:28 1,950 doubao.o
2024/11/05 23:01 2,968 test04.txt
2024/11/12 11:23 1,518 xunfei.c
2024/11/12 11:24 57,164 xunfei.exe
2024/11/12 11:24 2,040 xunfei.o
```

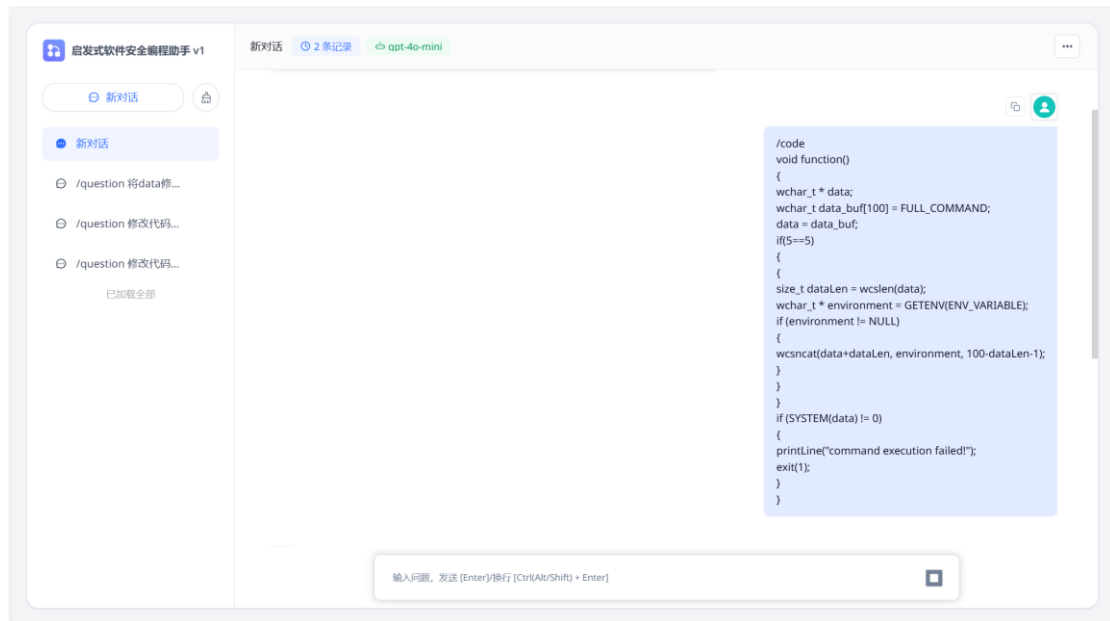
```
D:\SoftwareSecurity\homewo x + v
2024/11/12 11:27 1,346 chatgpt.c
2024/11/12 11:27 55,791 chatgpt.exe
2024/11/12 11:27 1,831 chatgpt.o
2024/11/12 11:23 3,193 CWE78_OS_Command_Injection__wchar_t_environment_system_03.c
2024/11/12 11:28 57,582 CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe
2024/11/12 11:28 2,879 CWE78_OS_Command_Injection__wchar_t_environment_system_03.o
2024/11/12 11:26 1,600 doubao.c
2024/11/12 11:28 56,656 doubao.exe
2024/11/12 11:28 1,950 doubao.o
2024/11/05 23:01 2,968 test04.txt
2024/11/12 11:23 1,518 xunfei.c
2024/11/12 11:24 57,164 xunfei.exe
2024/11/12 11:24 2,040 xunfei.o
13 个文件 246,518 字节
2 个目录 98,941,857,792 可用字节
Finished good()
Calling bad()...
驱动器 D 中的卷是 Data
卷的序列号是 6E99-8D8D

D:\SoftwareSecurity\homework\hw2\test\test04 的目录
2024/11/12 11:28 <DIR> .
2024/11/08 15:53 <DIR> ..
2024/11/12 11:27 1,346 chatgpt.c
2024/11/12 11:27 55,791 chatgpt.exe
2024/11/12 11:27 1,831 chatgpt.o
2024/11/12 11:23 3,193 CWE78_OS_Command_Injection__wchar_t_environment_system_03.c
2024/11/12 11:28 57,582 CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe
2024/11/12 11:28 2,879 CWE78_OS_Command_Injection__wchar_t_environment_system_03.o
2024/11/12 11:26 1,600 doubao.c
2024/11/12 11:28 56,656 doubao.exe
2024/11/12 11:28 1,950 doubao.o
2024/11/05 23:01 2,968 test04.txt
2024/11/12 11:23 1,518 xunfei.c
2024/11/12 11:24 57,164 xunfei.exe
2024/11/12 11:24 2,040 xunfei.o
13 个文件 246,518 字节
2 个目录 98,941,857,792 可用字节
Finished bad()

Process returned 0 (0x0) execution time : 0.205 s
Press any key to continue.
```

(1) 启发式软件安全编程助手

测试输入：



测试输出：





结果分析：

第 2 点指出了命令注入（Command Injection）风险。

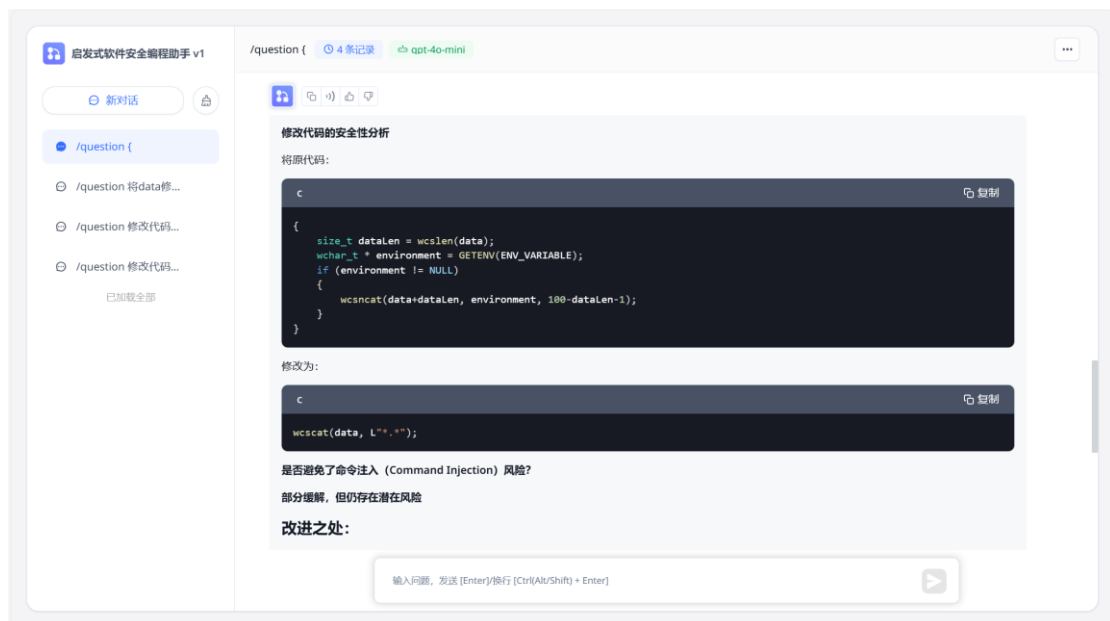
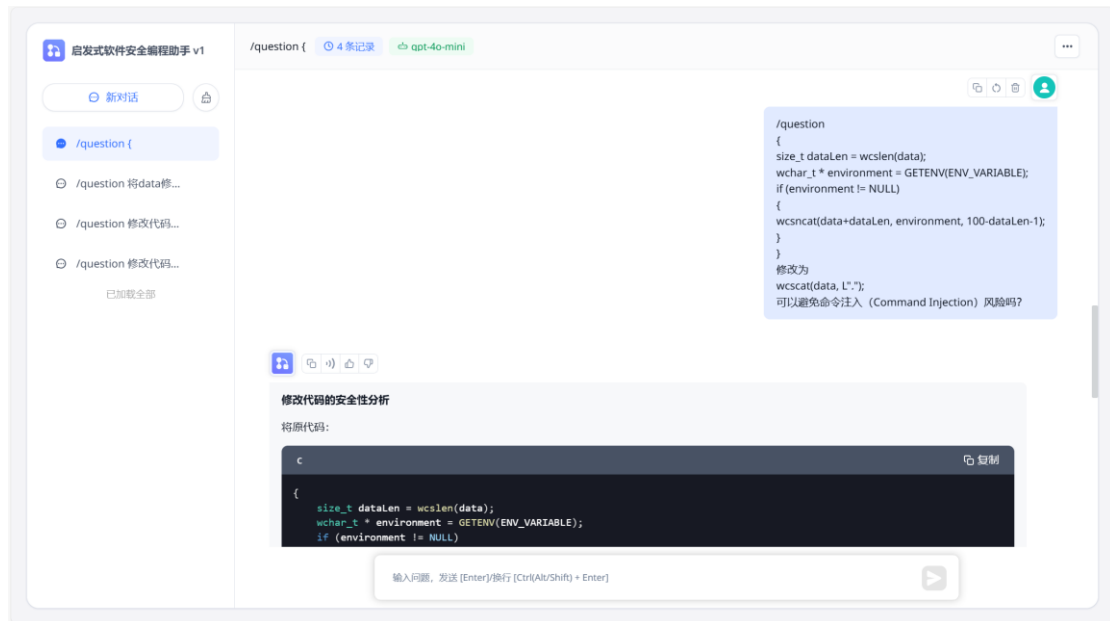
修改代码，将

```
{  
    size_t dataLen = wcslen(data);  
    wchar_t * environment = GETENV(ENV_VARIABLE);  
    if (environment != NULL)  
    {  
        wcsncat(data+dataLen, environment, 100-dataLen-1);  
    }  
}
```

修改为：

```
wcscat(data, L"*.");
```

询问这样的修改是否能避免这个问题：





模型回复能减少了可控输入, 降低了注入恶意命令风险。同时指出了其他风险, 如命令执行的上下文等。

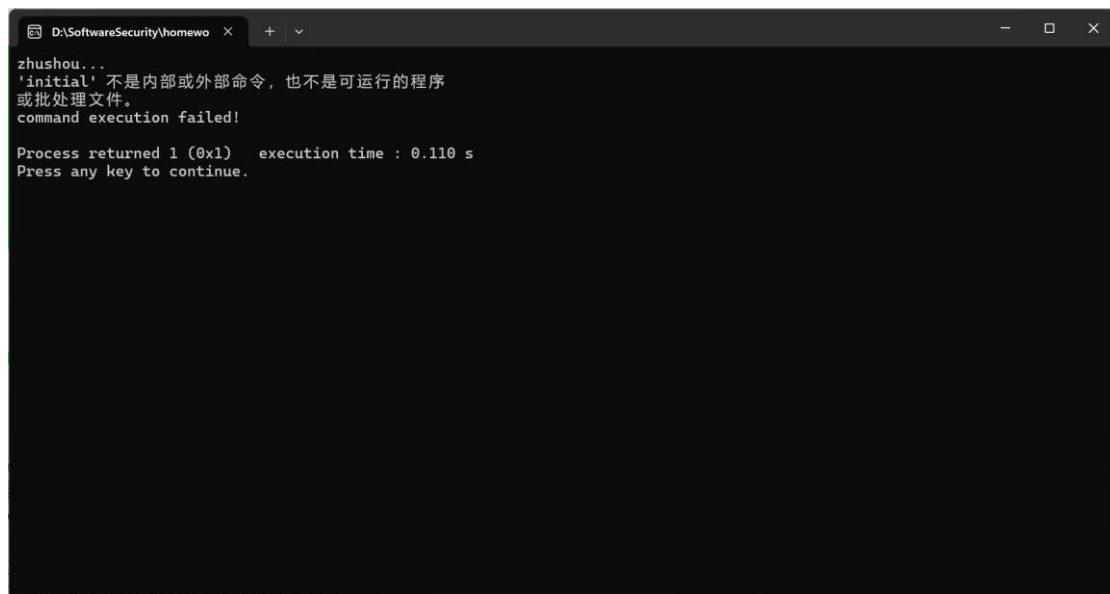
模型修改的代码:

```
void function()  
{  
    wchar_t * data;  
    wchar_t data_buf[100] = FULL_COMMAND;  
    data = data_buf;  
    if(5==5)  
    {  
        wcsncpy(data, L"*.*)" ;  
    }  
}
```

```
}  
if (SYSTEM(data) != 0)  
{  
    printLine("command execution failed!");  
    exit(1);  
}  
}
```

代码运行结果：运行 test04/zhushou.c:

运行结果错误。



The screenshot shows a Windows command prompt window with the title bar 'D:\SoftwareSecurity\homewo'. The command prompt displays the following text:

```
zhushou...  
'initial' 不是内部或外部命令，也不是可运行的程序  
或批处理文件。  
command execution failed!  
  
Process returned 1 (0x1)   execution time : 0.110 s  
Press any key to continue.
```

(2) ChatGPT

测试输入:



请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    wchar_t * data;
    wchar_t data_buf[100] = FULL_COMMAND;
    data = data_buf;
    if($==5)
    {
        {
            size_t dataLen = wcslen(data);
            wchar_t * environment = GETENV(ENV_VARIABLE);
            if (environment != NULL)
            {
                wcsncat(data+dataLen, environment, 100-dataLen-1);
            }
        }
    }
    if (SYSTEM(data) != 0)
    {
        printLine("command execution failed!");
        exit(1);
    }
}
```



给“ChatGPT”发送消息



ChatGPT 也可能犯错。请核查重要信息。



测试输出：



此代码存在安全漏洞，特别是**命令注入漏洞**。以下是漏洞行及原因分析：

漏洞分析

- 第10行: `wcsncat(data+dataLen, environment, 100-dataLen-1);`
 - 原因: 从环境变量获取的内容直接附加到命令字符串 `data` 上，而没有对内容进行任何验证或过滤。环境变量可能包含恶意内容，使得整个字符串 `data` 中的命令可以被攻击者操控。这将导致潜在的命令注入风险。
- 第14行: `SYSTEM(data);`
 - 原因: `SYSTEM` 函数直接执行由 `data` 指定的命令字符串。因此，如果 `data` 被恶意修改，攻击者可以通过该函数执行任意命令。

修复建议

- 使用白名单或过滤机制验证环境变量的内容，防止其包含不受信任的字符或命令。
- 如果可行，避免使用 `SYSTEM` 函数。可以考虑使用更安全的 API，如 `CreateProcess` (Windows) 或 `execve` (Linux) 等。
- 只允许预定义的命令执行，避免将用户输入直接拼接到命令中。

修复后的代码

以下是修复后的代码，通过移除环境变量的拼接来避免命令注入问题：

```
c
```



复制代码

给“ChatGPT”发送消息



ChatGPT 也可能犯错。请核查重要信息。





结果分析:

该模型找到了测试代码的命令注入漏洞并提供了漏洞所在行:

```
wcsncat(data+dataLen, environment, 100-dataLen-1);  
SYSTEM(data);
```

模型修改的代码:

```
void function()  
{  
    wchar_t * data;  
    wchar_t data_buf[100] = FULL_COMMAND;
```

```

data = data_buf;
if(5==5)
{
    // 移除对环境变量的读取
    printLine("Environment variable usage removed for security.");
}
// 限制 data 的内容为安全命令
if (wcscmp(data, FULL_COMMAND) == 0) // 假设 FULL_COMMAND 是一个安全的命令
{
    if (SYSTEM(data) != 0)
    {
        printLine("command execution failed!");
        exit(1);
    }
}
else
{
    printLine("Untrusted command, execution aborted.");
}
}

```

代码运行结果：运行 test04/chatgpt.c:

运行结果正确。

```

chatgpt...
Environment variable usage removed for security.
驱动器 D 中的卷是 Data
卷的序列号是 6E99-8D8D

D:\SoftwareSecurity\homework\hw2\test\test04 的目录
2024/11/12 11:27 <DIR>      .
2024/11/08 15:53 <DIR>      ..
2024/11/12 11:27          1,346 chatgpt.c
2024/11/12 11:27        55,791 chatgpt.exe
2024/11/12 11:27          1,831 chatgpt.o
2024/11/12 11:23          3,193 CWE78_OS_Command_Injection__wchar_t_environment_system_03.c
2024/11/12 11:23        57,582 CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe
2024/11/12 11:23          2,879 CWE78_OS_Command_Injection__wchar_t_environment_system_03.o
2024/11/12 11:26          1,600 doubao.c
2024/11/12 11:26        56,656 doubao.exe
2024/11/12 11:26          1,950 doubao.o
2024/11/05 23:01          2,968 test04.txt
2024/11/12 11:23          1,518 xunfei.c
2024/11/12 11:24          57,164 xunfei.exe
2024/11/12 11:24          2,040 xunfei.o
          13 个文件          246,518 字节
           2 个目录 98,941,857,792 可用字节

Process returned 0 (0x0)   execution time : 0.148 s
Press any key to continue.

```

5、CWE244_Heap_Inspection__w32_char_free_17.c

测试代码：

```

void function()
{
    int j;
    for(j = 0; j < 1; j++)
    {
        {
            char * password = (char *)malloc(100*sizeof(char));
            if (password == NULL) {exit(-1);}
            size_t passwordLen = 0;
            HANDLE hUser;
            char * username = "User";
            char * domain = "Domain";
            password[0] = '\\0';
            if (fgets(password, 100, stdin) == NULL)
            {
                printLine("fgets() failed");
                password[0] = '\\0';
            }
            passwordLen = strlen(password);
            if (passwordLen > 0)
            {
                password[passwordLen-1] = '\\0';
            }
            if (LogonUserA(
                username,
                domain,
                password,
                LOGON32_LOGON_NETWORK,
                LOGON32_PROVIDER_DEFAULT,
                &hUser) != 0)
            {
                printLine("User logged in successfully.");
                CloseHandle(hUser);
            }
            else
            {
                printLine("Unable to login.");
            }
            free(password);
        }
    }
}

```

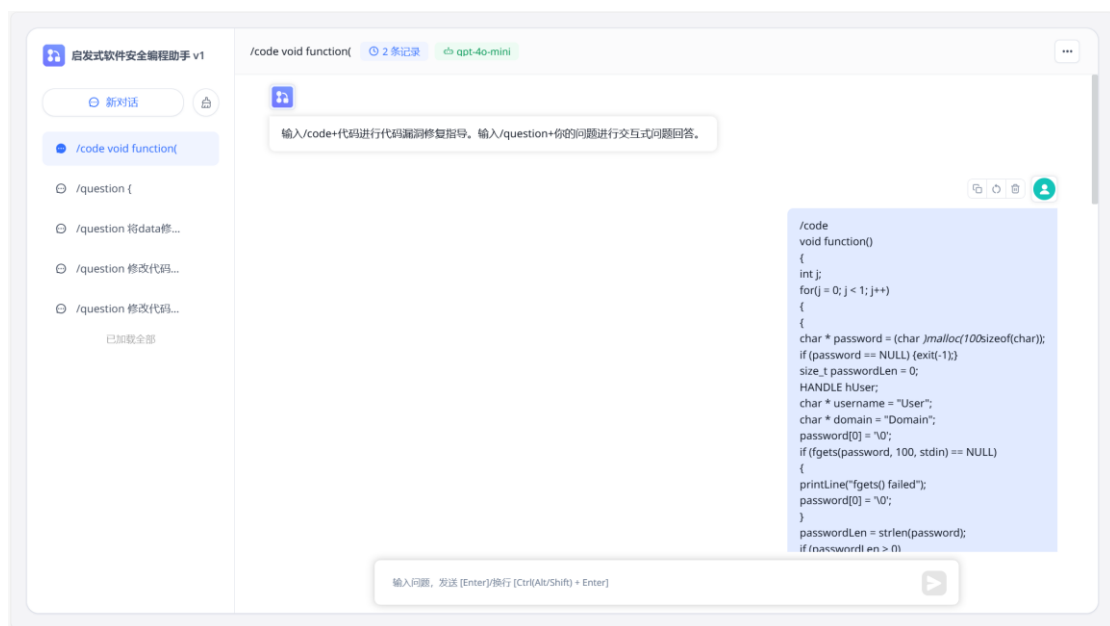
运行结果：运行 test05\CWE244_Heap_Inspection__w32_char_free_17.c:

```
D:\SoftwareSecurity\homewo x + v - □ x
Calling good()...
bupt
Unable to login.
Finished good()
Calling bad()...
bupt
Unable to login.
Finished bad()

Process returned 0 (0x0) execution time : 7.027 s
Press any key to continue.
```

(1) 启发式软件安全编程助手

测试输入：



测试输出：



结果分析:

第 3 点指出了错误处理和信息泄露风险。

修改代码, 在

```
free(password);
```

前加入:

```
passwordLen = strlen(password);
SecureZeroMemory(password, passwordLen * sizeof(char));
```

询问这样的修改是否能避免这个问题：



模型回复清除缓冲区,确实降低数据在内存中被意外访问或泄露的风险。并指出了其他风险,如 NULL 指针赋值等。

模型修改的代码:

```
void function()
{
    int j;
    for(j = 0; j < 1; j++)
    {
        {
```

```

char * password = (char *)malloc(100*sizeof(char));
if (password == NULL) {exit(-1);}
size_t passwordLen = 0;
HANDLE hUser;
char * username = "User";
char * domain = "Domain";
password[0] = '\0';
if (fgets(password, 100, stdin) == NULL)
{
    printLine("fgets() failed");
    password[0] = '\0';
}
passwordLen = strlen(password);
if (passwordLen > 0)
{
    password[passwordLen-1] = '\0';
}
if (LogonUserA(
    username,
    domain,
    password,
    LOGON32_LOGON_NETWORK,
    LOGON32_PROVIDER_DEFAULT,
    &hUser) != 0)
{
    printLine("User logged in successfully.");
    CloseHandle(hUser);
}
else
{
    printLine("Unable to login.");
}
passwordLen = strlen(password);
SecureZeroMemory(password, passwordLen * sizeof(char));
free(password);
}
}
}

```

代码运行结果：运行 test05/zhushou.c:

运行结果正确。

```
D:\SoftwareSecurity\homewo x + v
zhushou...
bupt
Unable to login.

Process returned 0 (0x0)   execution time : 6.350 s
Press any key to continue.
```

(2) ChatGPT

测试输入:



```
printf("get password\n");
password[0] = '\0';
}
passwordLen = strlen(password);
if (passwordLen > 0)
{
    password[passwordLen-1] = '\0';
}
if (LogonUserA(
    username,
    domain,
    password,
    LOGON32_LOGON_NETWORK,
    LOGON32_PROVIDER_DEFAULT,
    &hUser) != 0)
{
    printf("User logged in successfully.");
    CloseHandle(hUser);
}
else
{
    printf("Unable to login.");
    free(password);
}
}
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

?

测试输出：

此代码存在以下安全漏洞，特别是敏感数据泄露风险和内存管理问题。下面是漏洞分析及修复方法。

漏洞分析

- 第9行: `char * password = (char *)malloc(100*sizeof(char));`
 - 原因: 分配的 `password` 缓冲区用于存储敏感信息(如密码)。但是,在密码验证后,内存只是被 `free(password)` 释放,而密码数据依然可能残留在内存中,造成敏感数据泄露的风险。
- 第16行: `fgets(password, 100, stdin);`
 - 原因: `fgets` 从标准输入读取密码,但没有确保密码的长度不超过缓冲区大小。在实际使用中,如果输入过长,可能会被截断,或者潜在地导致缓冲区溢出,尽管这里限制了 100 字节,但最好显式保护读取的字节数。
- 缺少密码清除: 在 `free(password)` 前没有对密码进行安全清除,导致密码在内存释放前依然存在。

修复建议

- 清除密码缓冲区: 在 `free(password)` 之前使用 `memset` 函数清除密码,以确保密码不会残留在内存中。
- 改进输入读取: 使用安全的输入方法,如检查输入长度,避免密码被截断或溢出。
- 避免明文存储敏感数据: 尽量避免在内存中以明文形式存储敏感信息。如果可以,使用更加安全的密码处理方式。

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

?

修复后的代码

以下是改进后的代码:

```
c
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void function()
{
    int j;
    for(j = 0; j < 1; j++)
    {
        char * password = (char *)malloc(100*sizeof(char));
        if (password == NULL) {exit(-1);}
        size_t passwordLen = 0;
        HANDLE hUser;
        char * username = "User";
        char * domain = "Domain";
        password[0] = '\0';

        // 限制输入长度, 防止溢出
        if (fgets(password, 100, stdin) == NULL)
        {
            return;
        }
    }
}
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

```
// 限制输入长度, 防止溢出
if (fgets(password, 100, stdin) == NULL)
{
    printf("fgets() failed");
    password[0] = '\0';
}

passwordLen = strlen(password);
if (passwordLen > 0)
{
    password[passwordLen-1] = '\0'; // 去除末尾换行符
}

// 尝试登录
if (LogonUserA(
    username,
    domain,
    password,
    LOGON32_LOGON_NETWORK,
    LOGON32_PROVIDER_DEFAULT,
    &hUser) != 0)
{
    printf("User logged in successfully.");
    CloseHandle(hUser);
}
else
{
    return;
}
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析：

该模型找到了测试代码的敏感数据泄露风险和内存管理问题并提供了漏洞所在行：

```
char * password = (char *)malloc(100*sizeof(char));  
fgets(password, 100, stdin);
```

模型修改的代码：

```
#include <windows.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
void function()  
{  
    int j;  
    for(j = 0; j < 1; j++)  
    {  
        {  
            char * password = (char *)malloc(100*sizeof(char));  
            if (password == NULL) {exit(-1);}  
            size_t passwordLen = 0;  
            HANDLE hUser;  
            char * username = "User";  
            char * domain = "Domain";  
            password[0] = '\\0';
```

```

// 限制输入长度, 防止溢出
if (fgets(password, 100, stdin) == NULL)
{
    printLine("fgets() failed");
    password[0] = '\0';
}

passwordLen = strlen(password);
if (passwordLen > 0)
{
    password[passwordLen-1] = '\0'; // 去除末尾换行符
}

// 尝试登录
if (LogonUserA(
    username,
    domain,
    password,
    LOGON32_LOGON_NETWORK,
    LOGON32_PROVIDER_DEFAULT,
    &hUser) != 0)
{
    printLine("User logged in successfully.");
    CloseHandle(hUser);
}
else
{
    printLine("Unable to login.");
}

// 在释放前清除密码缓冲区
memset(password, 0, passwordLen);
free(password);
}
}
}

```

代码运行结果：运行 test05/chatgpt.c：

运行结果正确。

```
D:\SoftwareSecurity\homewo x + v
chatgpt...
bupt
Unable to login.

Process returned 0 (0x0)   execution time : 3.386 s
Press any key to continue.
```

6、CWE253_Incorrect_Check_of_Function_Return_Value__char_fgets_13.c

测试代码：

```
void function()
{
    if(GLOBAL_CONST_FIVE==5)
    {
        {
            char dataBuffer[100] = "";
            char * data = dataBuffer;
            printLine("Please enter a string: ");
            if (fgets(data, 100, stdin) < 0)
            {
                printLine("fgets failed!");
                exit(1);
            }
            printLine(data);
        }
    }
}
```

运行结果：运行

test06\CWE253_Incorrect_Check_of_Function_Return_Value__char_fgets_13.c:

```
D:\SoftwareSecurity\homewo x + v - □ ×
Calling good()...
Please enter a string:
bupt
bupt

Please enter a string:
bupt
bupt

Finished good()
Calling bad()...
Please enter a string:
bupt
bupt

Finished bad()

Process returned 0 (0x0)   execution time : 8.620 s
Press any key to continue.
|
```

(1) 启发式软件安全编程助手

测试输入：



测试输出：



结果分析：

第 3 点指出了错误处理问题。

修改代码，将

```
if (fgets(data, 100, stdin) < 0)
```

修改为：

```
if (fgets(data, 100, stdin) == NULL)
```

询问这样的修改是否能避免这个问题：



模型回复部分避免。并指出了仍需考虑是否需要区分错误、EOF 等。

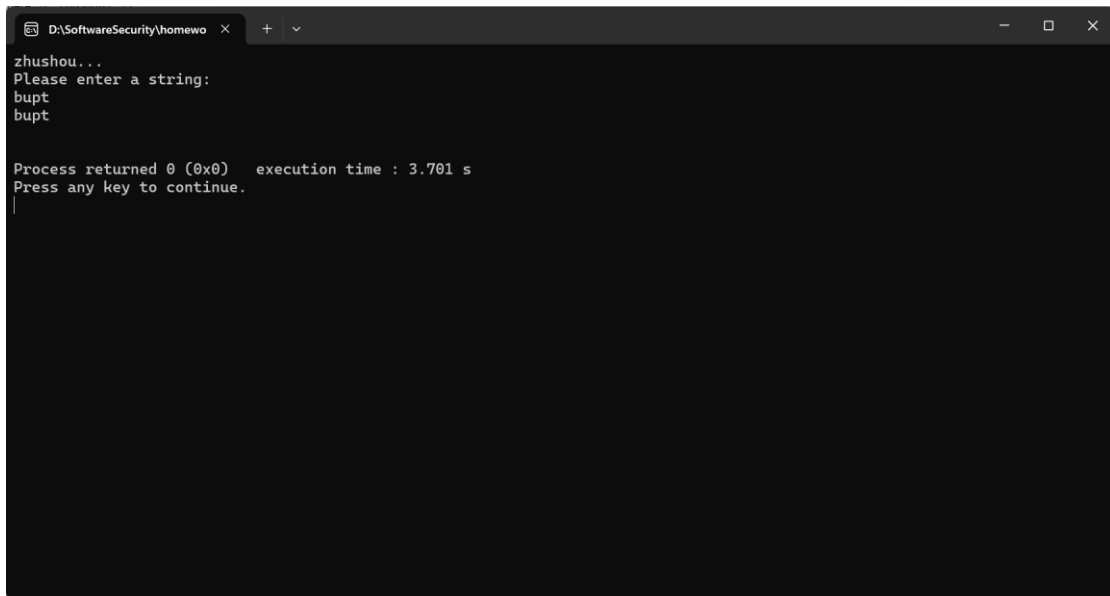
模型修改的代码:

```
void function()
{
    if(GLOBAL_CONST_FIVE==5)
    {
        {
            char dataBuffer[100] = "";
            char * data = dataBuffer;
            printLine("Please enter a string: ");
            if (fgetc(data, 100, stdin) == NULL)
```

```
    {  
        printLine("fgets failed!");  
        exit(1);  
    }  
    printLine(data);  
}  
}  
}
```

代码运行结果：运行 test06/zhushou.c:

运行结果正确。



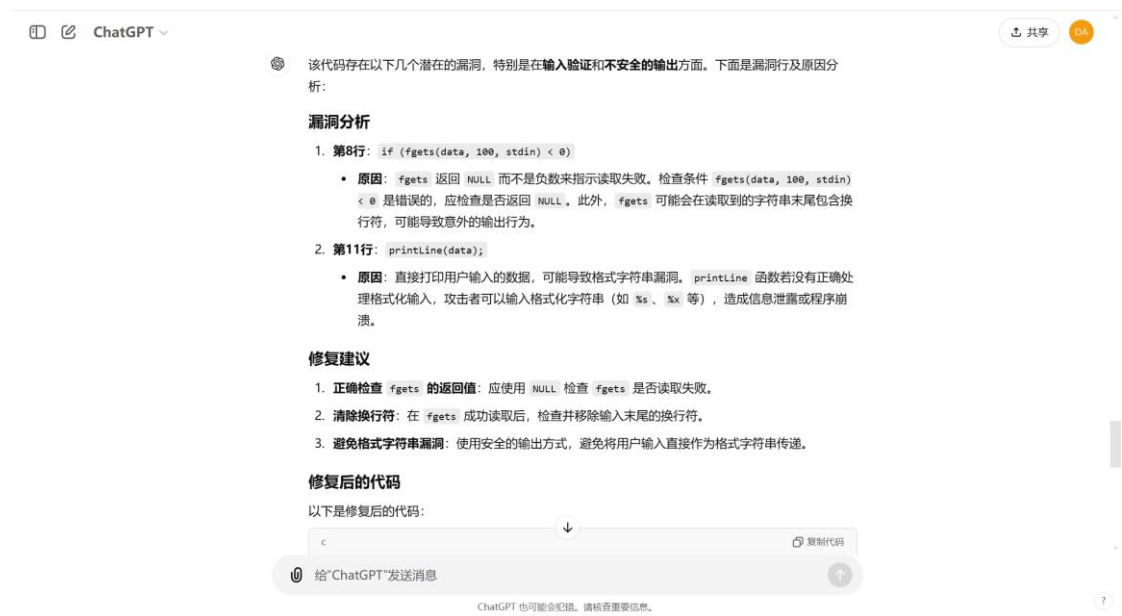
```
D:\SoftwareSecurity\homewo x + v  
zhushou...  
Please enter a string:  
bupt  
bupt  
  
Process returned 0 (0x0) execution time : 3.701 s  
Press any key to continue.
```

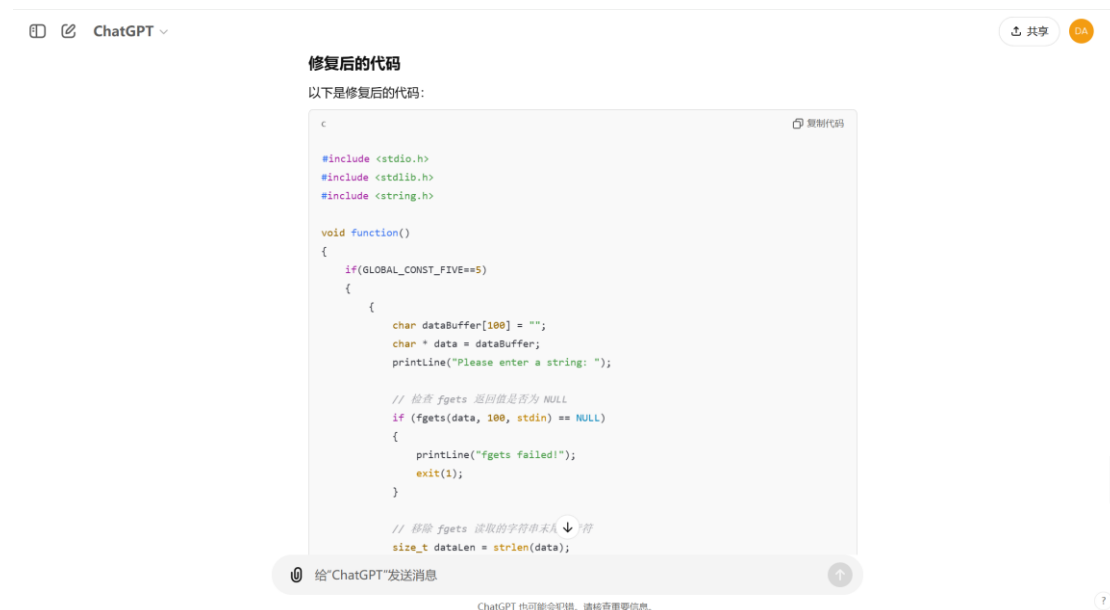
(2) ChatGPT

测试输入:



测试输出:





结果分析:

该模型找到了测试代码的输入验证和不安全的输出漏洞并提供了漏洞所在行:

```
if (fgets(data, 100, stdin) < 0)

printf(data);
```

模型修改的代码:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```

void function()
{
    if(GLOBAL_CONST_FIVE==5)
    {
        {
            char dataBuffer[100] = "";
            char * data = dataBuffer;
            printLine("Please enter a string: ");

            // 检查 fgets 返回值是否为 NULL
            if (fgets(data, 100, stdin) == NULL)
            {
                printLine("fgets failed!");
                exit(1);
            }

            // 移除 fgets 读取的字符串末尾换行符
            size_t dataLen = strlen(data);
            if (dataLen > 0 && data[dataLen - 1] == '\n')
            {
                data[dataLen - 1] = '\0';
            }

            // 使用安全的输出方式，避免格式字符串漏洞
            printf("%s\n", data);
        }
    }
}

```

代码运行结果：运行 test06/chatgpt.c：

运行结果正确。

```
D:\SoftwareSecurity\homewo x + v - □ x
chatgpt...
Please enter a string:
bupt
bupt

Process returned 0 (0x0)   execution time : 2.595 s
Press any key to continue.
```

7、CWE284_Improper_Access_Control__w32_char_CreateDesktop_11.c

测试代码：

```
void function()
{
    if(globalReturnsTrue())
    {
        {
            HDESK hDesk;
            char * desktopName = "DesktopExample";
            hDesk = CreateDesktopA(
                desktopName,
                NULL,
                NULL,
                0,
                GENERIC_ALL,
                NULL);

            if (hDesk == NULL)
            {
                printLine("Desktop could not be created");
            }
            else
            {
                printLine("Desktop created successfully");
                CloseDesktop(hDesk);
            }
        }
    }
}
```

```
}
```

运行结果：运行

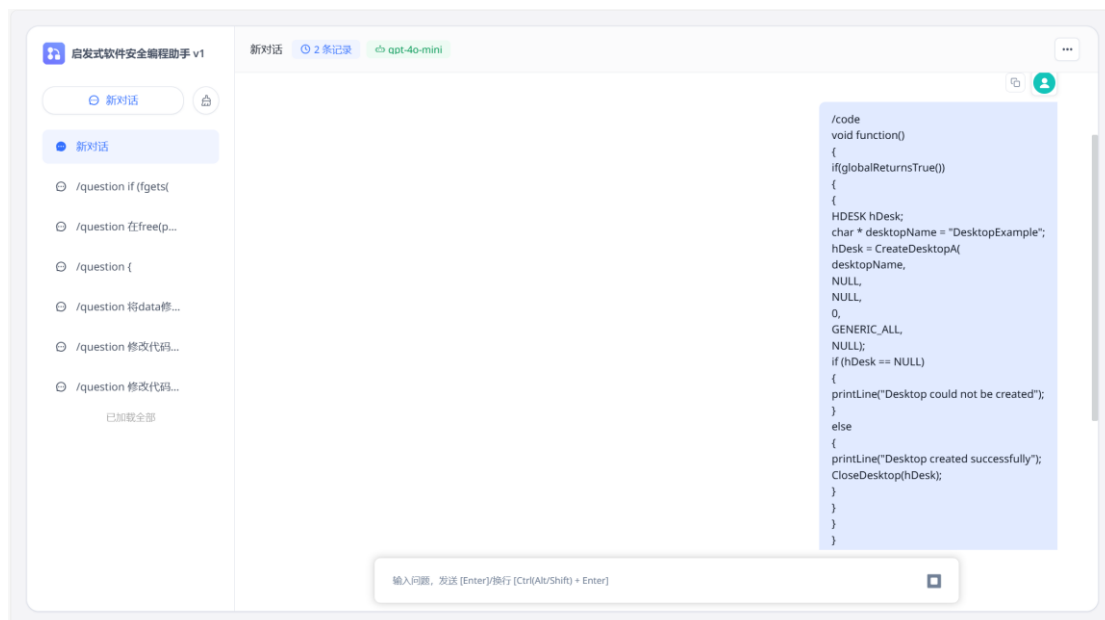
test07\CWE284_Improper_Access_Control__w32_char_CreateDesktop_11.c:

```
D:\SoftwareSecurity\homewo x + v -
Calling good()...
Desktop could not be created
Desktop could not be created
Finished good()
Calling bad()...
Desktop created successfully
Finished bad()

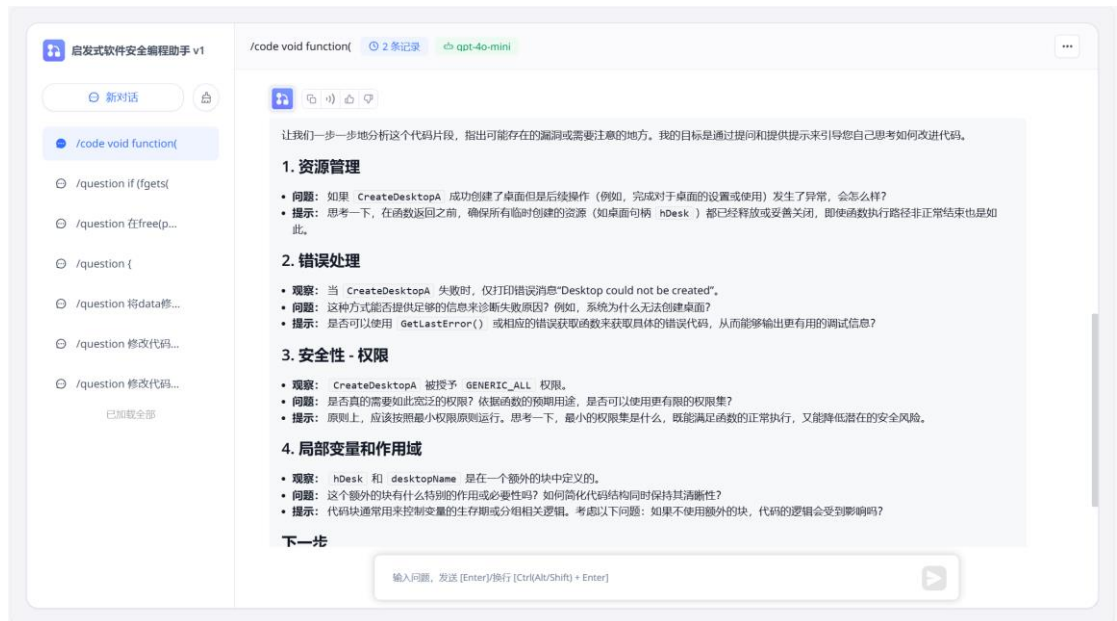
Process returned 0 (0x0)   execution time : 0.099 s
Press any key to continue.
```

(1) 启发式软件安全编程助手

测试输入：



测试输出：



结果分析：

第 3 点指出了安全性 - 权限问题。

修改代码，将

GENERIC_ALL

修改为：

GENERIC_READ

询问这样的修改是否能避免这个问题：



模型回复减少权限，进而降低风险。并提出了其他问题和提示。

模型修改的代码：

```
void function()  
{  
    if(globalReturnsTrue())  
    {  
        {  
            HDESK hDesk;  
            char * desktopName = "DesktopExample";  
            hDesk = CreateDesktopA(  
                desktopName,
```

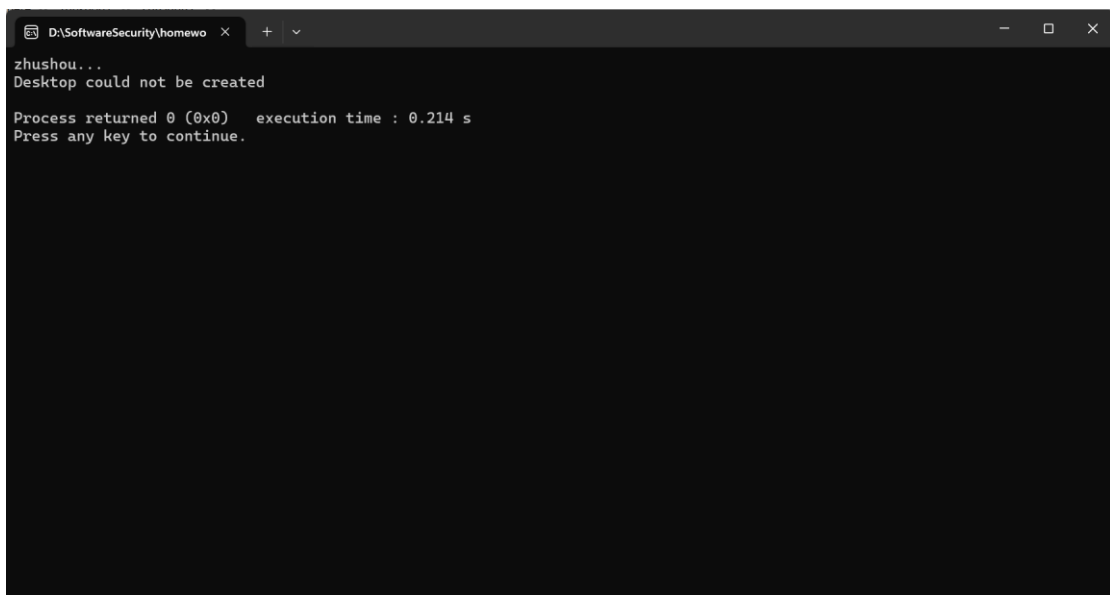
```

        NULL,
        NULL,
        0,
        GENERIC_READ,
        NULL);
    if (hDesk == NULL)
    {
        printLine("Desktop could not be created");
    }
    else
    {
        printLine("Desktop created successfully");
        CloseDesktop(hDesk);
    }
}
}
}

```

代码运行结果：运行 test07/zhushou.c:

运行结果与 good()一致，修改正确。



```

D:\SoftwareSecurity\homewo > zhushou...
Desktop could not be created

Process returned 0 (0x0)   execution time : 0.214 s
Press any key to continue.

```

(2) ChatGPT

测试输入:

请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    if(globalReturnsTrue())
    {
        {
            HDESK hDesk;
            char * desktopName = "DesktopExample";
            hDesk = CreateDesktopA(
                desktopName,
                NULL,
                NULL,
                0,
                GENERIC_ALL,
                NULL);
            if (hDesk == NULL)
            {
                printLine("Desktop could not be created");
            }
            else
            {
                printLine("Desktop created successfully");
                CloseDesktop(hDesk);
            }
        }
    }
}
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

测试输出：

该代码存在以下安全性和资源管理上的漏洞，尤其在**权限控制**和**资源清理**方面存在风险。下面是漏洞行及原因分析。

漏洞分析

- 第8行: `hDesk = CreateDesktopA(desktopName, NULL, NULL, 0, GENERIC_ALL, NULL);`
 - 原因: 调用 `CreateDesktopA` 并请求 `GENERIC_ALL` 权限。这会授予该桌面所有权限，可能被恶意软件利用来执行任意代码、篡改应用程序的界面、甚至导致系统不稳定。通常不需要这么高的权限，使用最小权限原则是更安全的选择。
- 第13行: `CloseDesktop(hDesk);`
 - 原因: 如果代码中 `CloseDesktop(hDesk);` 执行失败，资源将不会正确释放，可能导致句柄泄漏。应检查 `CloseDesktop` 的返回值并在失败时适当处理，以确保资源释放。

修复建议

- 降低权限: 仅请求创建桌面所需的最小权限，以防止不必要的访问风险。
- 检查 `CloseDesktop` 的返回值: 在关闭句柄时检查其返回值，确保资源被正确释放，防止句柄泄漏。

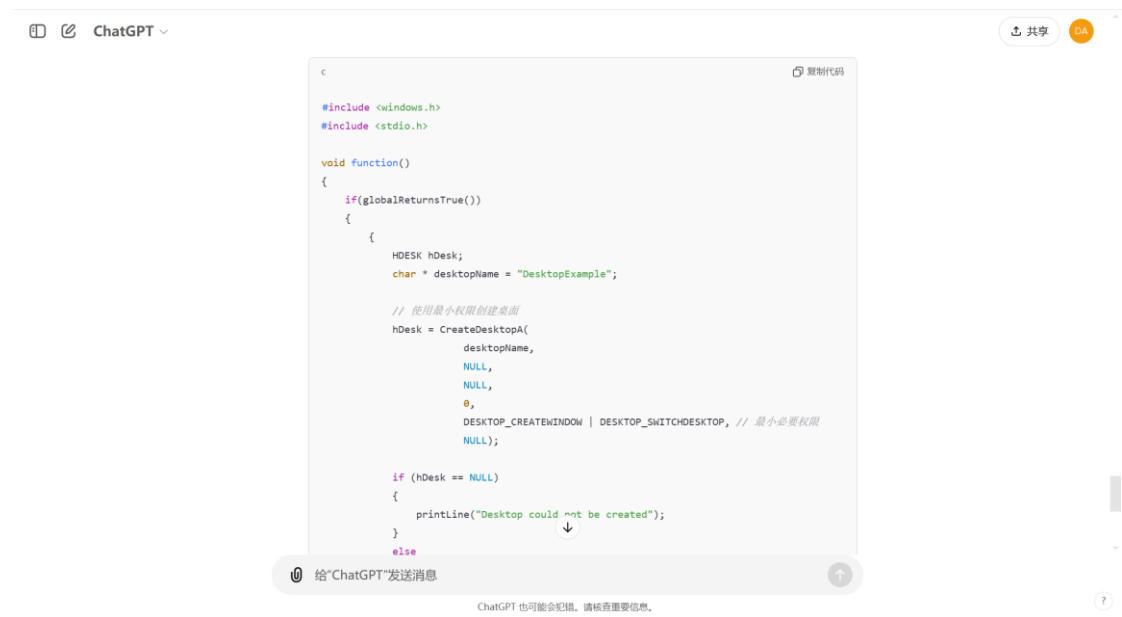
修复后的代码

以下是修复后的代码：

```
c
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析：

该模型找到了测试代码的权限控制和资源清理漏洞并提供了漏洞所在行：

```
hDesk = CreateDesktopA(desktopName, NULL, NULL, 0, GENERIC_ALL, NULL);  
CloseDesktop(hDesk);
```

模型修改的代码：

```
#include <windows.h>  
#include <stdio.h>  
  
void function()
```

```

{
    if(globalReturnsTrue())
    {
        {
            HDESK hDesk;
            char * desktopName = "DesktopExample";

            // 使用最小权限创建桌面
            hDesk = CreateDesktopA(
                desktopName,
                NULL,
                NULL,
                0,
                DESKTOP_CREATEWINDOW | DESKTOP_SWITCHDESKTOP, // 最小必要权限
                NULL);

            if (hDesk == NULL)
            {
                printLine("Desktop could not be created");
            }
            else
            {
                printLine("Desktop created successfully");

                // 确保句柄被正确关闭，避免泄漏
                if (!CloseDesktop(hDesk))
                {
                    printLine("Failed to close the desktop handle");
                }
            }
        }
    }
}

```

代码运行结果：运行 test07/chatgpt.c:

运行结果与 bad()一致。

```
D:\SoftwareSecurity\homewo x + v
xunfei...
Desktop created successfully

Process returned 0 (0x0)   execution time : 0.088 s
Press any key to continue.
```

8、CWE327_Use_Broken_Crypto__w32_DES_11.c

测试代码：

```
void function()
{
    if(globalReturnsTrue())
    {
        {
            FILE *pFile;
            HCRYPTPROV hCryptProv;
            HCRYPTKEY hKey;
            HCRYPTHASH hHash;
            char password[100];
            size_t passwordLen;
            char toBeDecrypted[100];
            DWORD toBeDecryptedLen = sizeof(toBeDecrypted)-1;
            printLine("Enter the password: ");
            if (fgets(password, 100, stdin) == NULL)
            {
                printLine("fgets() failed");
                password[0] = '\0';
            }
            passwordLen = strlen(password);
            if (passwordLen > 0)
            {
                password[passwordLen-1] = '\0';
            }
            pFile = fopen("encrypted.txt", "rb");
        }
    }
}
```

```

    if (pFile == NULL)
    {
        exit(1);
    }
    if (fread(toBeDecrypted, sizeof(char), 100, pFile) != 100)
    {
        fclose(pFile);
        exit(1);
    }
    toBeDecrypted[99] = '\0';
    if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV, PROV_RSA_AES,
0))
    {
        if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV,
PROV_RSA_AES, CRYPT_NEWKEYSET))
        {
            printLine("Error in acquiring cryptographic context");
            exit(1);
        }
    }
    if(!CryptCreateHash(hCryptProv, CALG_SHA_256, 0, 0, &hHash))
    {
        printLine("Error in creating hash");
        exit(1);
    }
    if(!CryptHashData(hHash, (BYTE *) password, passwordLen, 0))
    {
        printLine("Error in hashing password");
        exit(1);
    }
    if(!CryptDeriveKey(hCryptProv, CALG_DES, hHash, 0, &hKey))
    {
        printLine("Error in CryptDeriveKey");
        exit(1);
    }
    if(!CryptDecrypt(hKey, 0, 1, 0, (BYTE *)toBeDecrypted, &toBeDecryptedLen))
    {
        printLine("Error in decryption");
        exit(1);
    }
    toBeDecrypted[toBeDecryptedLen] = '\0';
    printLine(toBeDecrypted);
    if (hKey)
    {

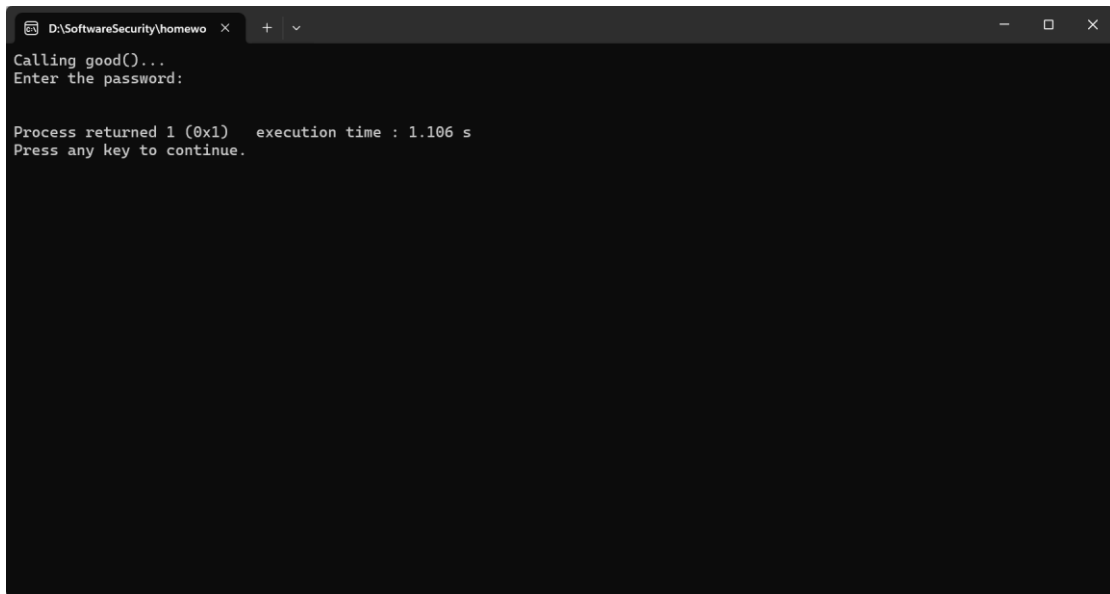
```

```

        CryptDestroyKey(hKey);
    }
    if (hHash)
    {
        CryptDestroyHash(hHash);
    }
    if (hCryptProv)
    {
        CryptReleaseContext(hCryptProv, 0);
    }
    if (pFile)
    {
        fclose(pFile);
    }
}
}
}

```

运行结果：运行 test08\CWE327_Use_Broken_Crypto__w32_DES_11.c:



```

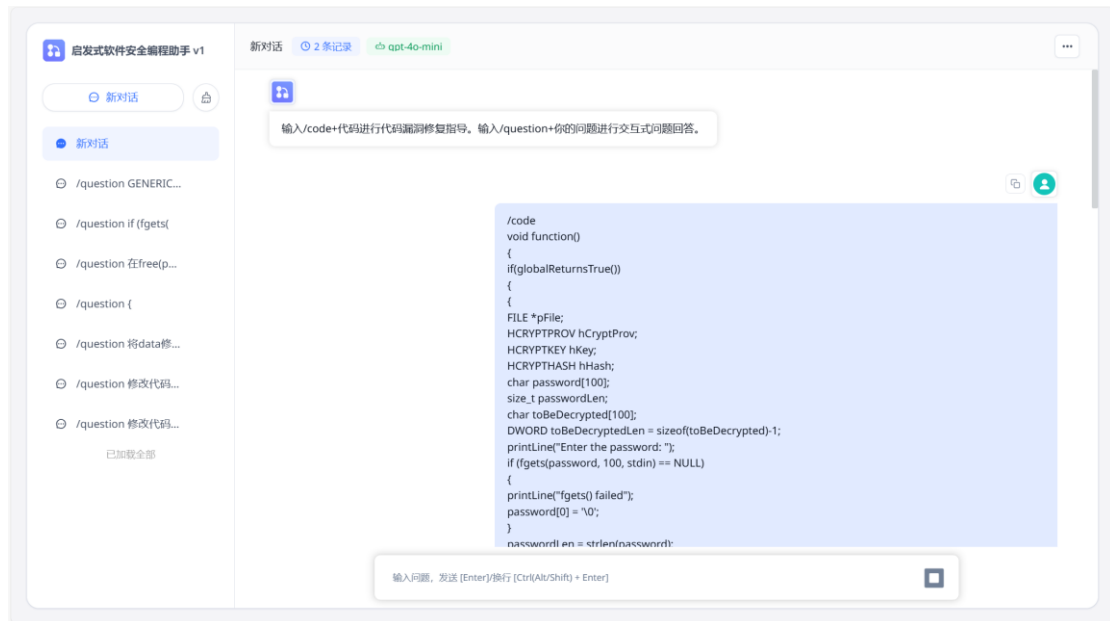
D:\SoftwareSecurity\homewo >
Calling good()...
Enter the password:

Process returned 1 (0x1)   execution time : 1.106 s
Press any key to continue.

```

(1) 启发式软件安全编程助手

测试输入:



测试输出：





结果分析：

第 3 点指出了使用的算法 DES 可能不满足安全需求，考虑新的或被广泛接受的最佳实践（例如，使用 AES 加密）。

修改代码，将 DES 算法修改为 AES。

询问这样的修改是否能避免这个问题：





模型替换加密算法（从 DES 到 AES）足以避免不安全的加密问题。并指出应考虑 AES 模式、密钥管理等其他问题。

模型修改的代码：

```
void function()
{
    if(globalReturnsFalse())
    {
        printLine("Benign, fixed string");
    }
    else
    {
        {
            FILE *pFile;
            HCRYPTPROV hCryptProv;
            HCRYPTKEY hKey;
            HCRYPTHASH hHash;
            char password[100];
            size_t passwordLen;
            char toBeDecrypted[100];
            DWORD toBeDecryptedLen = sizeof(toBeDecrypted)-1;
            printLine("Enter the password: ");
            if (fgets(password, 100, stdin) == NULL)
            {
                printLine("fgets() failed");
                password[0] = '\0';
            }
        }
    }
}
```

```

passwordLen = strlen(password);
if (passwordLen > 0)
{
    password[passwordLen-1] = '\\0';
}
pFile = fopen("encrypted.txt", "rb");
if (pFile == NULL)
{
    exit(1);
}
if (fread(toBeDecrypted, sizeof(char), 100, pFile) != 100)
{
    fclose(pFile);
    exit(1);
}
toBeDecrypted[99] = '\\0';
if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV, PROV_RSA_AES,
0))
{
    if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV,
PROV_RSA_AES, CRYPT_NEWKEYSET))
    {
        printLine("Error in acquiring cryptographic context");
        exit(1);
    }
}
if(!CryptCreateHash(hCryptProv, CALG_SHA_256, 0, 0, &hHash))
{
    printLine("Error in creating hash");
    exit(1);
}
if(!CryptHashData(hHash, (BYTE *) password, passwordLen, 0))
{
    printLine("Error in hashing password");
    exit(1);
}
if(!CryptDeriveKey(hCryptProv, CALG_AES_256, hHash, 0, &hKey))
{
    printLine("Error in CryptDeriveKey");
    exit(1);
}
if(!CryptDecrypt(hKey, 0, 1, 0, (BYTE *)toBeDecrypted, &toBeDecryptedLen))
{
    printLine("Error in decryption");
}

```

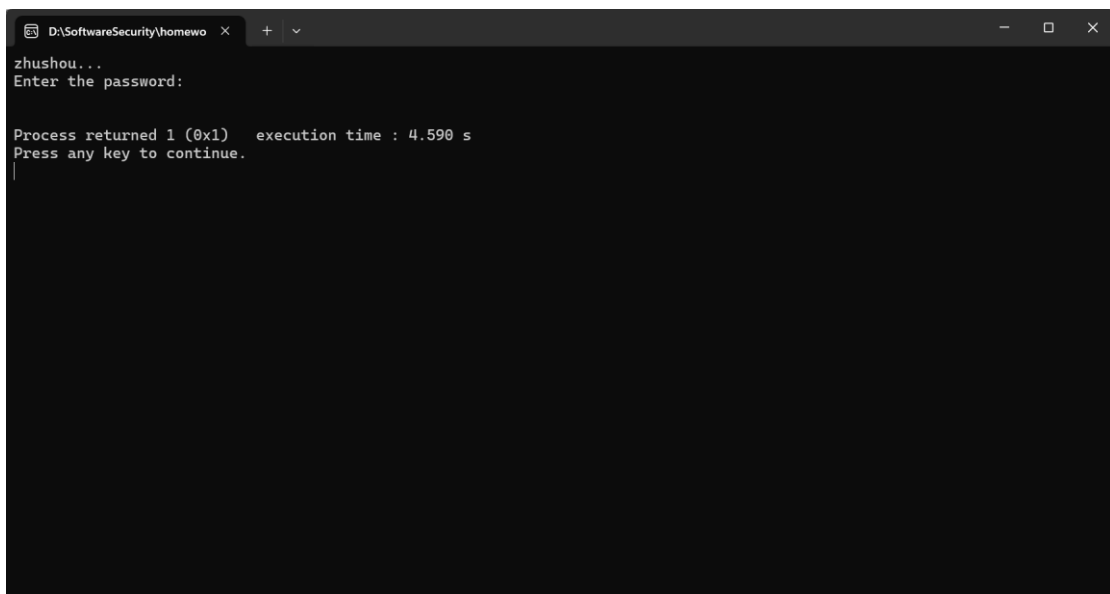
```

        exit(1);
    }
    toBeDecrypted[toBeDecryptedLen] = '\0';
    printLine(toBeDecrypted);
    if (hKey)
    {
        CryptDestroyKey(hKey);
    }
    if (hHash)
    {
        CryptDestroyHash(hHash);
    }
    if (hCryptProv)
    {
        CryptReleaseContext(hCryptProv, 0);
    }
    if (pFile)
    {
        fclose(pFile);
    }
}
}
}

```

代码运行结果：运行 test08/zhushou.c:

修改正确。



```

D:\SoftwareSecurity\homewo x + v
zhushou...
Enter the password:

Process returned 1 (0x1) execution time : 4.590 s
Press any key to continue.

```

(2) ChatGPT

测试输入：

🔍

📄

ChatGPT

▼

🔗 共享

👤 DA

🗑

给“ChatGPT”发送消息

↑

🔍

ChatGPT 也可能犯错。请核查重要信息。

?

🗑

给你检查以下代码是否存在漏洞，如果存在请标出漏洞行号，并给出修复后的代码

void function()
{
 if(globalReturnsTrue())
 {
 {
 FILE *pFile;
 HCRYPTPROV hCryptProv;
 HCRYPTKEY hKey;
 HCRYPTHASH hHash;
 char password[100];
 size_t passwordLen;
 char toBeDecrypted[100];
 DWORD toBeDecryptedLen = sizeof(toBeDecrypted)-1;
 println("Enter the password: ");
 if (fgets(password, 100, stdin) == NULL)
 {
 println("fgets() failed");
 password[0] = '\0';
 }
 passwordLen = strlen(password);
 if (passwordLen > 0)
 {
 password[passwordLen-1] = '\0';
 ↓
 pFile = fopen("encrypted.txt", "r");
 }
 }
 }
}

测试输出：

🔍

📄

ChatGPT

▼

🔗 共享

👤 DA

🗑

给“ChatGPT”发送消息

↑

🔍

ChatGPT 也可能犯错。请核查重要信息。

?

🗑

此代码存在以下漏洞，主要涉及敏感信息清理、错误处理和资源管理。下面是漏洞行及原因分析。

漏洞分析

1. 第14行: fgets(password, 100, stdin);

原因:

从标准输入读取密码后未清理，释放资源后可能在内存中残留敏感信息，导致密码泄露风险。

2. 第24行: if (fread(toBeDecrypted, sizeof(char), 100, pFile) != 100)

原因:

fread 未正确读取到 100 字节数据时直接退出，未考虑文件可能小于 100 字节。此操作会导致未完整读取内容的情况下程序退出，可能影响程序逻辑。

3. 第29行: CryptAcquireContext

原因:

CryptAcquireContext 和后续加密函数未考虑资源的分配与释放顺序，一旦函数执行失败会直接 exit(1)，可能导致句柄泄漏。此外，若无法获取上下文，系统会尝试创建新密钥集，但可能没有必要。

4. 缺少密码清理:

敏感数据 password 和 toBeDecrypted 在使用完成后应当被安全清理，避免内存中保留明文信息。

修复建议

1. 清理敏感数据:

在释放资源和退出前，使用 memset 清理 password 和 toBeDecrypted 的内容。

2. 文件读取的长度检查:

在读取文件时动态 ↓ 数据长度，避免固定长度读取的潜在错误。



结果分析：

该模型找到了测试代码的敏感信息清理、错误处理和资源管理漏洞并提供了漏洞所在行：

```
fgets(password, 100, stdin);

if (fread(toBeDecrypted, sizeof(char), 100, pFile) != 100)

CryptAcquireContext
```

正确漏洞修复方法应是用 AES 替代 DES，但该模型并未如此做。

模型修改的代码：

```

#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void function()
{
    if(globalReturnsTrue())
    {
        FILE *pFile = NULL;
        HCRYPTPROV hCryptProv = 0;
        HCRYPTKEY hKey = 0;
        HCRYPTHASH hHash = 0;
        char password[100];
        size_t passwordLen;
        char toBeDecrypted[100] = {0}; // 初始化为零
        DWORD toBeDecryptedLen = sizeof(toBeDecrypted)-1;

        printLine("Enter the password: ");
        if (fgets(password, 100, stdin) == NULL)
        {
            printLine("fgets() failed");
            password[0] = '\0';
            return;
        }

        // 去除末尾换行符
        passwordLen = strlen(password);
        if (passwordLen > 0)
        {
            password[passwordLen-1] = '\0';
        }

        pFile = fopen("encrypted.txt", "rb");
        if (pFile == NULL)
        {
            printLine("File open failed.");
            memset(password, 0, sizeof(password)); // 清理密码
            return;
        }

        size_t bytesRead = fread(toBeDecrypted, sizeof(char), sizeof(toBeDecrypted)-1,
pFile);
        if (bytesRead == 0)
    }

```

```

{
    printLine("File read failed.");
    fclose(pFile);
    memset(password, 0, sizeof(password)); // 清理密码
    return;
}
toBeDecrypted[bytesRead] = '\0';

if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV, PROV_RSA_AES, 0))
{
    if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV, PROV_RSA_AES,
CRYPT_NEWKEYSET))
    {
        printLine("Error in acquiring cryptographic context");
        memset(password, 0, sizeof(password)); // 清理密码
        fclose(pFile);
        return;
    }
}

if(!CryptCreateHash(hCryptProv, CALG_SHA_256, 0, 0, &hHash))
{
    printLine("Error in creating hash");
    memset(password, 0, sizeof(password));
    CryptReleaseContext(hCryptProv, 0);
    fclose(pFile);
    return;
}

if(!CryptHashData(hHash, (BYTE *) password, passwordLen, 0))
{
    printLine("Error in hashing password");
    memset(password, 0, sizeof(password));
    CryptDestroyHash(hHash);
    CryptReleaseContext(hCryptProv, 0);
    fclose(pFile);
    return;
}

if(!CryptDeriveKey(hCryptProv, CALG_DES, hHash, 0, &hKey))
{
    printLine("Error in CryptDeriveKey");
    memset(password, 0, sizeof(password));
    CryptDestroyHash(hHash);

```

```

        CryptReleaseContext(hCryptProv, 0);
        fclose(pFile);
        return;
    }

    if(!CryptDecrypt(hKey, 0, TRUE, 0, (BYTE *)toBeDecrypted, &toBeDecryptedLen))
    {
        printLine("Error in decryption");
        memset(password, 0, sizeof(password));
        memset(toBeDecrypted, 0, sizeof(toBeDecrypted));
        CryptDestroyKey(hKey);
        CryptDestroyHash(hHash);
        CryptReleaseContext(hCryptProv, 0);
        fclose(pFile);
        return;
    }
    toBeDecrypted[toBeDecryptedLen] = '\0';
    printLine(toBeDecrypted);

    // 清理敏感数据并释放资源
    memset(password, 0, sizeof(password));
    memset(toBeDecrypted, 0, sizeof(toBeDecrypted));
    if (hKey)
    {
        CryptDestroyKey(hKey);
    }
    if (hHash)
    {
        CryptDestroyHash(hHash);
    }
    if (hCryptProv)
    {
        CryptReleaseContext(hCryptProv, 0);
    }
    if (pFile)
    {
        fclose(pFile);
    }
}
}

```

代码运行结果：运行 test08/chatgpt.c:

```
D:\SoftwareSecurity\homewo x + v
chatgpt...
Enter the password:
Error in decryption
Process returned 0 (0x0) execution time : 1.046 s
Press any key to continue.
```

9、CWE469_Use_of_Pointer_Subtraction_to_Determine_Size__char_14.c

测试代码：

```
void function()
{
    if(globalFive==5)
    {
        {
            char string1[] = SOURCE_STRING;
            char string2[] = SOURCE_STRING;
            char * slashInString1;
            size_t indexOfSlashInString1;
            slashInString1 = strchr(string1, '/');
            if (slashInString1 == NULL)
            {
                exit(1);
            }
            indexOfSlashInString1 = (size_t)(slashInString1 - string2);
            printUnsignedLine(indexOfSlashInString1);
        }
    }
}
```

运行结果：运行

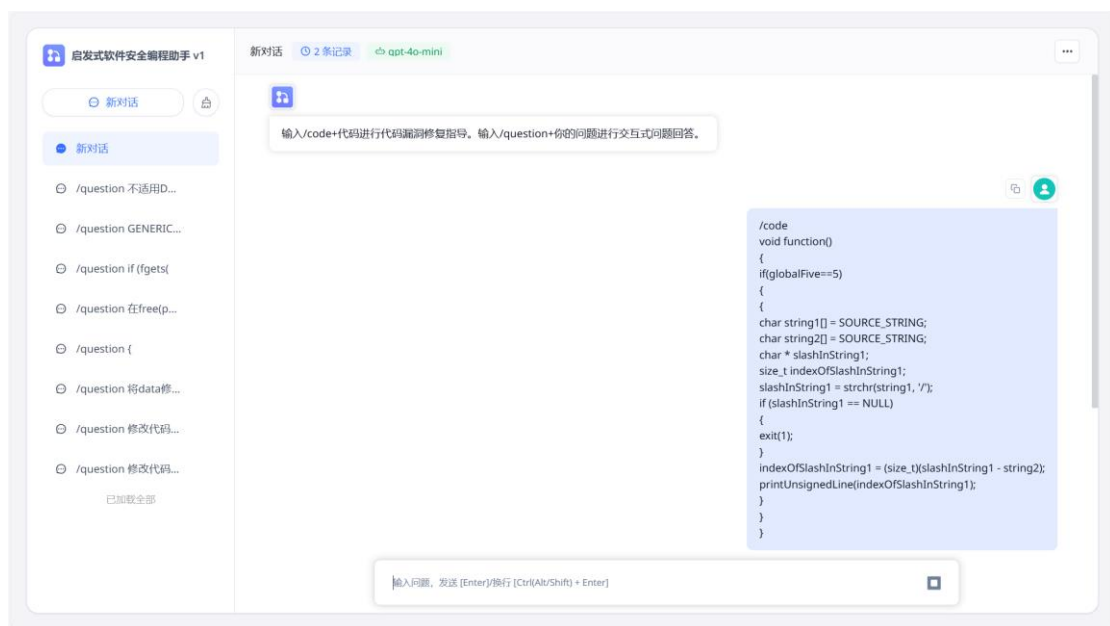
test09/CWE469_Use_of_Pointer_Subtraction_to_Determine_Size__char_14.c:

```
D:\SoftwareSecurity\homewo x + v - □ x
Calling good()...
3
3
Finished good()
Calling bad()...
15
Finished bad()

Process returned 0 (0x0)   execution time : 0.068 s
Press any key to continue.
```

(1) 启发式软件安全编程助手

测试输入：



测试输出：

启发式软件安全编程助手 v1

新对话

/code void function()

/question 不适用D...

/question GENERIC...

/question if (fgets(

/question in free(p...

/question {

/question 将data修...

/question 修改代码...

/question 修改代码...

已加载全部

/code void function()

2 条记录

qpt-4o-mini

...

让我们逐步分析您的代码，找出可能存在的漏洞或改进方向，以下是我的反馈，旨在指导您自主思考和解决问题：

1. 全局变量使用: globalFive

问题: 使用全局变量可能导致代码的可靠性和维护性降低，因为它们的状态可以从任何地方被修改。

提示: 思考一下，如果在这个例子中使用函数参数来传递值会怎样？这可能有助于提高代码的封装性。

2. 数组初始化

代码:

C

undefined

复制

char string1[] = SOURCE_STRING;

char string2[] = SOURCE_STRING;

复制

- **问题**：两个数组被赋予相同的值，但之后的操作似乎假设它们应该是不同的引用点（见下面的`strchr`和指针运算）。

- **提示**：您是否本意要赋予它们不同的值？如果是相同值的意图，请考虑为什么会有两个相同的变量，以及是否可以简化这些变量的使用。

3. **指针运算**

- **代码**:

```C

indexOfSlashInString1 = (size\_t)(slashInString1 - string2);

输入问题，发送 [Enter] 换行 [Ctrl(Alt/Shift) + Enter]

启发式软件安全编程助手 v1

新对话

/code void function()

/question 不适用D...

/question GENERIC...

/question if (fgets(

/question in free(p...

/question {

/question 将data修...

/question 修改代码...

/question 修改代码...

已加载全部

/code void function()

2 条记录

qpt-4o-mini

...

### 3. \*\*指针运算\*\*

- \*\*代码\*\*:

```C

indexOfSlashInString1 = (size_t)(slashInString1 - string2);

问题: 这里的指针运算是一个常见的陷阱。指针减法仅在两个指针指向同一个数组（或该数组的末尾元素之后的一个元素）的元素时才有定义的行为。这里诊断的是`string2`是否确实是您打算用来作为基准的数组。

提示: 并且考虑使用`strchr`返回的指针与`string1`（不是`string2`）进行运算是否更合适？如果您的目标是找到斜线在`string1`中的索引。

4. 错误处理

代码:

C

undefined

复制

if (slashInString1 == NULL)

{

exit(1);

}

复制

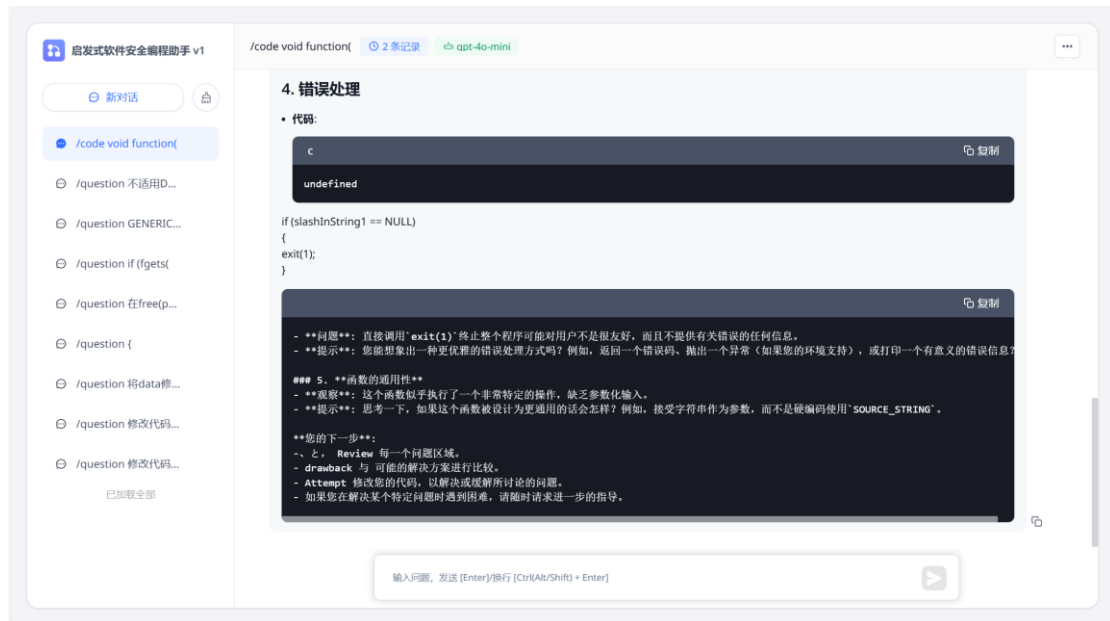
- **问题**：直接调用`exit(1)`终止整个程序可能对用户不是很友好，而且不提供有关错误的任何信息。

- **提示**：您能想象出一种更优雅的错误处理方式吗？例如，返回一个错误码、抛出一个异常（如果您的环境支持），或打印一个有意义的错误信息。

5. **函数的通用性**

输入问题，发送 [Enter] 换行 [Ctrl(Alt/Shift) + Enter]

78 / 92



结果分析：

第 3 点指出了指针运算问题。

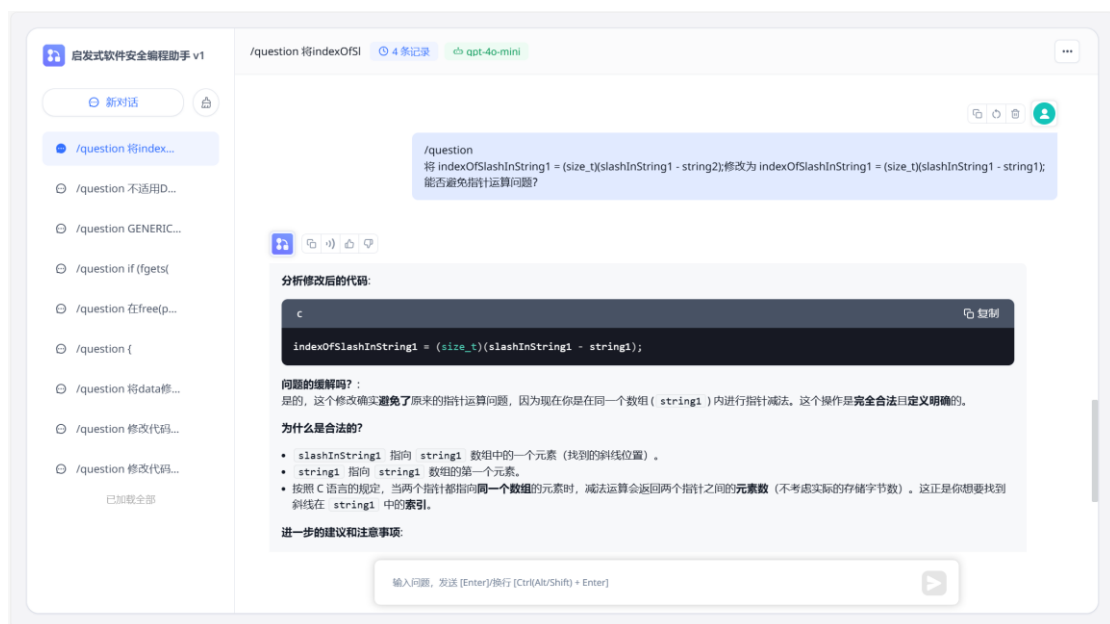
修改代码，将

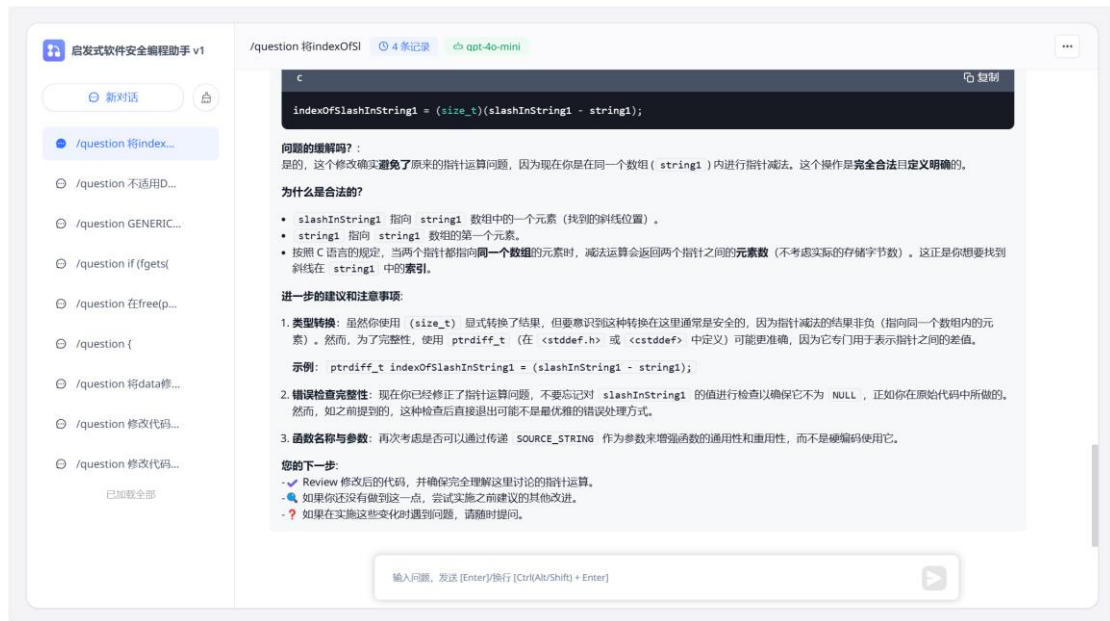
```
indexOfSlashInString1 = (size_t)(slashInString1 - string2);
```

修改为：

```
indexOfSlashInString1 = (size_t)(slashInString1 - string1);
```

询问这样的修改是否能避免这个问题：





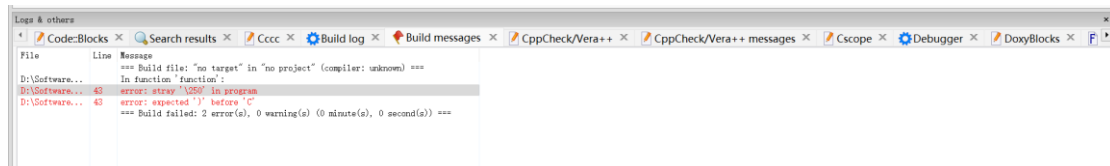
模型回复避免了指针运算问题。并给出了为什么正确的解释和进一步的建议与注意事项。

模型修改的代码：

```
void function()
{
    if(globalFive==5)
    {
        {
            char string1[] = SOURCE_STRING;
            char string2[] = SOURCE_STRING;
            char * slashInString1;
            size_t indexOfSlashInString1;
            slashInString1 = strchr(string1, '/');
            if (slashInString1 == NULL)
            {
                exit(1);
            }
            indexOfSlashInString1 = (size_t)(slashInString1 - string1);
            printUnsignedLine(indexOfSlashInString1);
        }
    }
}
```

代码运行结果：运行 test09/zhushou.c:

运行失败。



(2) ChatGPT

测试输入:



测试输出:





结果分析：

该模型找到了测试代码的缓冲区溢出漏洞并提供了漏洞所在行：

```
indexOfSlashInString1 = (size_t)(slashInString1 - string2);
```

模型修改的代码：

```
void function()
{
    if(globalFive == 5)
    {
        {
            char string1[] = SOURCE_STRING;
            char string2[] = SOURCE_STRING;
            char *slashInString1;
            size_t indexOfSlashInString1;
            slashInString1 = strchr(string1, '/');
            if (slashInString1 == NULL)
            {
                exit(1);
            }
            indexOfSlashInString1 = (size_t)(slashInString1 - string1); // 修复行
            printUnsignedLine(indexOfSlashInString1);
        }
    }
}
```

代码运行结果：运行 test09/chatgpt.c：

运行结果正确，与 good()一致。

A screenshot of a Windows command prompt window. The title bar shows the path 'D:\SoftwareSecurity\homewo'. The command prompt displays the output of a program: 'chatgpt...' followed by '3', then 'Process returned 0 (0x0) execution time : 0.067 s' and 'Press any key to continue.'

10、CWE511_Logic_Time_Bomb__counter_16.c

测试代码：

```
void function()
{
    while(1)
    {
        {
            int count = 0;
            do
            {
                if (count == COUNT_CHECK)
                {
                    UNLINK("important_file.txt");
                }
                count++;
            }
            while(1 == 1);
        }
        break;
    }
}
```

| | | | |
|---|------------------|------|------|
|  important_file1.txt | 2024/11/12 14:18 | 文本文档 | 0 KB |
|  important_file2.txt | 2024/11/12 14:18 | 文本文档 | 0 KB |
|  important_file3.txt | 2024/11/12 14:18 | 文本文档 | 0 KB |
|  important_file4.txt | 2024/11/12 14:18 | 文本文档 | 0 KB |

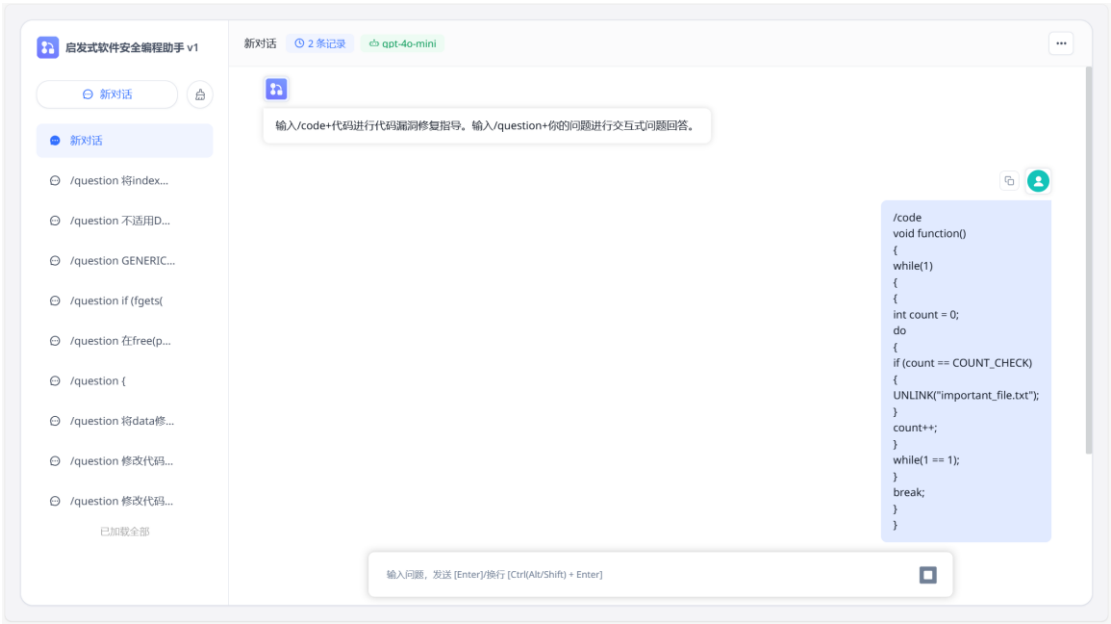
运行结果：运行 test10/ CWE511_Logic_Time_Bomb__counter_16.c:

important_file1.txt 已被删除。

| | | | |
|---|------------------|------|------|
|  important_file2.txt | 2024/11/12 14:18 | 文本文档 | 0 KB |
|  important_file3.txt | 2024/11/12 14:18 | 文本文档 | 0 KB |
|  important_file4.txt | 2024/11/12 14:18 | 文本文档 | 0 KB |

(1) 启发式软件安全编程助手

测试输入：



测试输出：



结果分析：

第 4 点指出了安全性 - 文件删除问题。

修改代码，将

```
UNLINK("important_file.txt");
```

修改为：

```
println("GOOD!");
```

询问这样的修改是否能避免这个问题：



模型回复避免了文件删除问题。但要考虑程序意图和行为，是否符合设计目标和要求等。

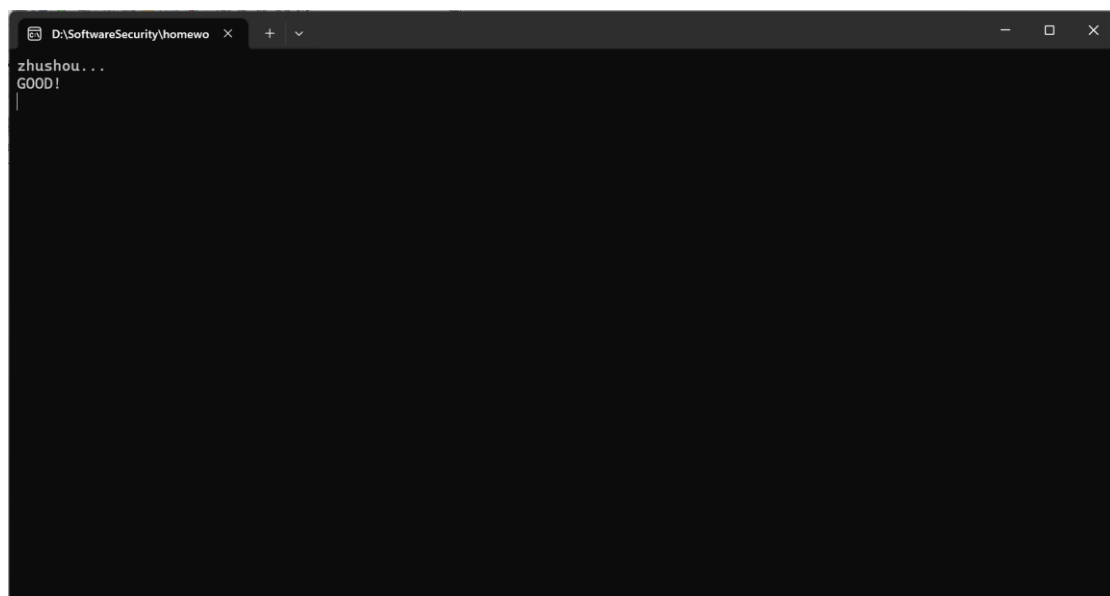
模型修改的代码：

```
void function()
{
    while(1)
    {
        {
            int count = 0;
            do
            {
```

```
        if (count == COUNT_CHECK)
        {
            printLine("GOOD!");
        }
        count++;
    }
    while(1 == 1);
}
break;
}
}
```

代码运行结果：运行 test10/zhushou.c:

important_file2.txt 未被删除。



(2) ChatGPT

测试输入:

请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    while(1)
    {
        {
            int count = 0;
            do
            {
                if (count == COUNT_CHECK)
                {
                    UNLINK("important_file.txt");
                }
                count++;
            }
            while(1 == 1);
        }
        break;
    }
}
```

这段代码中存在一个漏洞：

给“ChatGPT”发消息

ChatGPT 也可能犯错。请核查重要信息。

测试输出：

这段代码中存在一个漏洞：

漏洞行号：

```
C
while(1 == 1);
```

原因： 这里的 `while(1 == 1);` 形成了一个无限循环，因为 `1 == 1` 始终为真，这会导致程序在该循环中不断执行下去。在 `count` 达到 `COUNT_CHECK` 时将会调用 `UNLINK("important_file.txt")` 删除文件 `important_file.txt`。由于 `count` 不会被重置，此循环会继续删除文件而不停止，可能导致文件意外删除或资源浪费。

修复后的代码： 应修改 `while(1 == 1)` 以提供适当的循环退出条件，例如设定上限或根据某种条件终止循环。以下代码示例使用 `break` 来退出循环：

```
C
void function()
{
    while(1)
    {
        {
            int count = 0;
            do
            {
```

给“ChatGPT”发消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析:

该模型找到了测试代码的无限循环漏洞并提供了漏洞所在行:

```
while(1 == 1);
```

模型修改的代码:

```
void function()
{
    while(1)
    {
        {
            int count = 0;
            do
            {
                if (count == COUNT_CHECK)
                {
                    UNLINK("important_file.txt");
                    break; // 修复行: 退出循环
                }
                count++;
            }
            while(count < COUNT_CHECK + 1); // 修复行: 限定循环条件
        }
        break;
    }
}
```

代码运行结果：运行 test10/chatgpt.c：

important_file4.txt 已被删除。

二、分析

| 测试用例 | 模型 | 是否找到代码漏洞 | 是否提供漏洞原因 | 代码是否运行 | 漏洞是否修复 | 备注 |
|--------|-------------|----------|----------|--------|--------|------------------------------------|
| TEST01 | 启发式软件安全编程助手 | 是 | 是 | 是 | 是 | |
| | ChatGPT | 是 | 是 | 是 | 是 | |
| TEST02 | 启发式软件安全编程助手 | 是 | 是 | 是 | 是 | |
| | ChatGPT | 是 | 是 | 否 | | |
| TEST03 | 启发式软件安全编程助手 | 是 | 是 | 是 | 是 | |
| | ChatGPT | 是 | 是 | 是 | 否 | 输出结果与 bad()一致，与 good()不一致，说明并未修复漏洞 |
| TEST04 | 启发式软件安全编程助手 | 是 | 是 | 否 | | |
| | ChatGPT | 是 | 是 | 是 | 是 | |
| TEST05 | 启发式软件安全编程助手 | 是 | 是 | 是 | 是 | |

| | | | | | | |
|--------|-------------|---|---|---|---|--------------------------------------|
| | ChatGPT | 是 | 是 | 是 | 是 | |
| TEST06 | 启发式软件安全编程助手 | 是 | 是 | 是 | 是 | |
| | ChatGPT | 是 | 是 | 是 | 是 | |
| TEST07 | 启发式软件安全编程助手 | 是 | 是 | 是 | 是 | 输出结果与 good() 一致，说明修复漏洞 |
| | ChatGPT | 是 | 是 | 是 | 否 | 输出结果与 bad() 一致，与 good() 不一致，说明并未修复漏洞 |
| TEST08 | 启发式软件安全编程助手 | 是 | 是 | 是 | 是 | 正确使用漏洞修复方法：用 AES 替代 DES |
| | ChatGPT | 是 | 是 | 是 | 否 | 正确漏洞修复方法应用 AES 替代 DES，但模型并未如此做 |
| TEST09 | 启发式软件安全编程助手 | 是 | 是 | 否 | | |
| | ChatGPT | 是 | 是 | 是 | 是 | |
| TEST10 | 启发式软件安全编程助手 | 是 | 是 | 是 | 是 | |
| | ChatGPT | 是 | 是 | 是 | 是 | |

1、测试说明

此次测试是基于上一次测试（第二次平时作业思考题部分）所做的。采用上次测试中表现最好的模型——ChatGPT。

2、结果分析

经过十个用例的测试，有以下的初步分析：

(1) 启发式软件安全编程助手的**引导能力很强**。在给它输入代码后，会进行详细的引导，告知漏洞类型、漏洞代码、如何修复，启发使用者思考。

(2) 启发式软件安全编程助手的**漏洞检测能力很强**。能发现所有测试用例的漏洞，并且能准确分析出漏洞原因和漏洞相关代码。

(3) 启发式软件安全编程助手的**漏洞修复能力很强**。在所有可以运行的测试用例中，均能正确修复漏洞。

3、测试结论

启发式软件安全编程助手相比于其他模型，最大的特点在于：不是单纯的输入问题输出答案模式，而是去引导使用者一步一步走向正确答案。给使用者提供详细的漏洞相关信息和下一步的操作。避免同学们简单粗暴的复制粘贴，这种“无脑”的方式，无疑是不利于学生的学业进步的。启发式软件安全编程助手采用“启发式”方案，像一位“24 小时在线的全能教师”，给同学们一个正确的可实施的前进方向，使同学们学会自主思考、自主探索。

4、改进建议

然而，在测试过程中，启发式软件安全编程助手暴露出了一些弊端。如，

(1) **同一时间只支持单个用户访问**。这是服务器算力不足的弊端，如后续大规模投入使用的话，建议采用算力更大的服务器。

(2) **有时会“胡言乱语”**。如下图，可能是模型不稳定等问题，目前尚不清楚具体原因。

